LOYOLA – ICAM COLLEGE OF ENGINEERING AND TECHNOLOGY (LICET)
Loyola campus, Nungambakkam, Chennai – 34

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CCS342 DEVOPS LAB MANUAL

SEMESTER: VI

Document No:	Date of Issue:	Compiled By:	Authorized By
LICET/CSE/LAB MANUAL/DEVOPS [CCS342]	18/01/2024	Ms J Limsa Joshi AP/CSE	
Version: 1.0	**Date of Revision:**	**Verified By:** Ms J Limsa Joshi AP/CSE	**HoD CSE**

ANNA UNIVERSITY, CHENNAI
NON - AUTONOMOUS AFFILIATED COLLEGES
REGULATIONS 2021

CCS342 DEVOPS Lab

Objectives:

- To introduce DevOps terminology, definition & concepts
- To understand the different Version control tools like Git, Mercurial
- To understand the concepts of Continuous Integration/ Continuous Testing/ Continuous Deployment
- To understand Configuration management using Ansible

List of Experiments

1. Identify a problem statement and write the software requirement specification
2. Project implementation using MVC and Spring Framework
3. Install Git version control system and configure with GitHub
4. Installation and build using Maven
5. Installation and build using Gradle
6. Create a Maven build pipeline in Azure
7. Run regression test using Maven build pipeline in Azure
8. Installation and Configuration of Jenkins and creating a workspace
9. Install Jenkins in cloud and create CI/CD pipeline and deploy in cloud
10. Install and configure Ansible and Create an Ansible playbook for a simple web application infrastructure

Outcomes:

- Understand different actions performed through Version control tools like Git. (K3)
- Perform Continuous Integration and Continuous Testing and Continuous Deployment using Jenkins by building and automating test cases using Maven & Gradle. (K3)
- Ability to Perform Automated Continuous Deployment. (K3)
- Ability to do Configuration Management using Ansible. (K3)
- Understand how to leverage Cloud-based DevOps tools using Azure DevOps. (K3)
- Design, develop, and deploy a full-stack application using MVC architecture with the Spring Boot framework, integrating version control, CI/CD, configuration management, and cloud-based DevOps tools. (K3)

CO-PO Mapping														
PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
3	2			3				2	2		2	3	2	
3	2	2		3						2	2	3	2	
3	2	2		3						2	2	3	2	
3	2	2		3						2	2	3	2	
3	2	2		3						2	3	3	2	
3	3	3	2	3				2	2	3	3	3	2	

Ex No. 1 Identify a problem statement and write the software requirement specification

Project Title: **Online Course Registration System**

1. Introduction

1.1 Purpose

The purpose of this document is to outline the software requirements for an Online Course Registration System (OCRS). This system will allow students to register for courses online, view available courses, and manage their course schedules. It will also provide administrative functionality for faculty and department heads to manage course offerings and student enrollments.

1.2 Scope

The OCRS will serve educational institutions by automating the student course registration process. The system will:

- Allow students to view and register for courses online.
- Allow administrators to add, update, and delete courses.
- Allow faculty to manage enrolled students.
- Ensure registration is only possible if prerequisites are met and seats are available.

The system will be accessible via web browsers and built using web technologies like HTML, CSS, JavaScript, and backend support using PHP/Python/Java with a MySQL/PostgreSQL database.

1.3 Definitions, Acronyms, and Abbreviations

- OCRS: Online Course Registration System
- UI: User Interface
- DBMS: Database Management System
- CRUD: Create, Read, Update, Delete

2. Overall Description

2.1 Product Perspective

The OCRS is a self-contained web application. It interacts with a central database storing all user, course, and registration data. It will be hosted on a cloud or on-premise server accessible via institution intranet or internet.

2.2 Product Functions

- Student login, profile management.
- View available courses and register.
- Add/Drop courses within the allowed period.
- Admin can manage courses, student accounts, and generate reports.
- Faculty can view student lists and post course details.

2.3 User Classes and Characteristics

- **Student:** End users who register for courses.
- **Faculty:** View students enrolled, manage course details.
- **Admin:** Manages users, courses, registration rules.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Student Module

- Register/Login using credentials.
- View list of available courses.
- Register/Drop courses.
- View course schedule and enrollment status.

3.1.2 Admin Module

- Login to the admin dashboard.
- Add/Edit/Delete course information.
- Approve student registrations.
- Generate reports on course enrollments.

3.1.3 Faculty Module

- View list of students enrolled in their course.
- Update course information like syllabus, schedules.

3.2 Non-Functional Requirements

3.2.1 Usability

- Simple and intuitive UI for ease of use.

3.2.2 Security

- Encrypted passwords and role-based access control.

3.2.3 Performance

- Handle at least 100 concurrent users.

3.2.4 Reliability

- 99.9% uptime during registration period.

3.2.5 Maintainability

- Modular architecture to allow easy updates.

Ex No. 2 Project Implementation using MVC and Spring Boot

Register.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <title>Register Page</title>
</head>
<body>
    <h1>Register</h1>

    <form th:action="@{/register}" method="post">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username" required /> <br /><br />

        <label for="password">Password:</label>
        <input type="password" id="password" name="password" required /> <br /><br />

        <input type="submit" value="Register" />
    </form>
        <p th:if="${error}" th:text="${error}" style="color:red;"></p>
    <div>
        <h3>Already have an account? <a th:href="@{/}">Login</a></h3>
    </div>
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Login page</title>
  </head>
  <body>
    <h1>Login page</h1>
     <form th:action="@{/login}" method="post">
      <label for="username">Username</label>:
      <input type="text" id="username" name="username" autofocus="autofocus" /> <br />
      <label for="password">Password</label>:
      <input type="password" id="password" name="password" /> <br />
      <input type="submit" value="Login" />
    </form>
    <p th:if="${error}" th:text="${error}" style="color: red;"></p>
        <div align="left">
        <h3><a th:href="@{/register}">Register</a></h3>
    </div>
      </body>
</html>
```

Welcome.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">

<head>
    <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootswatch/4.5.2/cosmo/bootstrap.min.css" />
    <script src= "https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"
></script>
</head>
<body>
<div>
    <h2 >Student Course Registation</h2>
    <tr>
        <div align = "left" >
             <h3><a  th:href="@{'/new'}">Add new</a></h3>
        </div>

    </tr>
    <tr>

    <div class="col-sm-5" align = "center">
                  <div class="panel-body" align = "center" >
                  <table class="table">
  <thead class="thead-dark">
    <tr>
            <th>Student ID</th>
            <th>Student Name</th>
            <th>Course</th>
            <th>Fee</th>
            <th>Edit</th>
            <th>Delete</th>
    </tr>
  </thead>
  <tbody>
      <tr  th:each="student : ${liststudent}">
        <td th:text="${student.id}">Student ID</td>
        <td th:text="${student.studentname}">StudentName</td>
        <td th:text="${student.course}">Course</td>
        <td th:text="${student.fee}">Fee</td>
        <td>
            <a th:href="@{'/edit/' + ${student.id}}">Edit</a>
        </td>
        <td>
            <a th:href="@{'/delete/' + ${student.id}}">Delete</a>
        </td>
        </tr>

  </tbody>
</table>
```

```
                </div>

            </div>

    </tr>

    </tbody>
    </table>
 <!-- <a th:href="@{/logout}">Logout</a>-->
 <form th:action="@{/logout}" method="get">
    <button type="submit">Logout</button>
</form>
</body>
</html>
```

New.html

```
<!DOCTYPE html>
<html xmlns=”http://www.w3.org/1999/xhtml”
      xmlns:th=”http://www.thymeleaf.org”>
<head>
    <meta charset=”utf-8” />
    <title>Create New Product</title>
    <link rel=”stylesheet”
href=”https://stackpath.bootstrapcdn.com/bootswatch/4.5.2/cosmo/bootstrap.min.css” />
    <script src= “https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js”
></script>
</head>
<body>
<div align=”center”>
    <h1>Create New Student</h1>
    <br />
     <div class=”col-sm-4”>
    <form action=”#” th:action=”@{/save}” th:object=”${student}” method=”post”>


        <div alight=”left”>
            <tr>
                <label class=”form-label” >Student Name</label>
                <td>
                    <input type=”hidden” th:field=”*{id}” />
                    <input type=”text” th:field=”*{studentname}” class=”form-control”
placeholder=”Student Name” />
                </td>
            </tr>
         </div>

            <div alight=”left”>

            <tr>
                 <label class=”form-label” >Course</label>
                <td><input type=”text” th:field=”*{course}” class=”form-control”
placeholder=”Course” /></td>
```

```
            </tr>
            </div>
            <div alight="left">
                 <tr>
                 <label class="form-label" >Fee</label>
                <td><input type="text" th:field="*{fee}" class="form-control" placeholder="Fee"
/></td>
            </tr>
            </div>
            <br>
            <tr>
            <td colspan="2"><button type="submit" class="btn btn-info">Save</button> </td>
            </tr>
        </table>
    </form>
</div>
</body>
</html>
```

Student.java

```
package com.batch2.artifact1.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "student")
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String studentname;
    private String course;
    private int fee;

    public Student() {
    }

    public Student(Long id, String studentname, String course, int fee) {
        this.id = id;
        this.studentname = studentname;
        this.course = course;
        this.fee = fee;
    }

    public Long getId() {
```

```
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getStudentname() {
        return studentname;
    }

    public void setStudentname(String studentname) {
        this.studentname = studentname;
    }

    public String getCourse() {
        return course;
    }

    public void setCourse(String course) {
        this.course = course;
    }

    public int getFee() {
        return fee;
    }

    public void setFee(int fee) {
        this.fee = fee;
    }
}
```

Login.java

```
package com.batch2.artifact1.domain;

import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "login")
public class Login {
    @Id
    public String username;
    public String password;

    public Login() {
    }

    public Login(String username, String password) {
        this.username = username;
        this.password = password;
```

```
    }

    public String getUsername() {
        return username;
    }

    public void setUsername(String username) {
        this.username = username;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }
}
```

RegRepo.java

```
package com.batch2.artifact1.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.batch2.artifact1.domain.Login;

@Repository
public interface RegRepo extends JpaRepository<Login, String> {
    Login findByUsername(String username);
}
```

LoginRepo.java

```
package com.batch2.artifact1.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.batch2.artifact1.domain.Login;

@Repository
public interface LoginRepo extends JpaRepository<Login, String> {
    Login findByUsernameAndPassword(String username, String password);
}
```

CourseRepo.java

```
package com.batch2.artifact1.repository;

import org.springframework.data.jpa.repository.JpaRepository;

import com.batch2.artifact1.domain.Student;
```

```
public interface CourseRepo extends JpaRepository<Student, Long> {

}
```

RegService.java

```
package com.batch2.artifact1.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.batch2.artifact1.domain.Login;
import com.batch2.artifact1.repository.RegRepo;

@Service
public class RegService {
    @Autowired
    private RegRepo rep;

    public boolean registerUser(String username, String password) {

        if (rep.findByUsername(username) != null) {
            return false;
        }

        Login newUser = new Login(username, password);
        rep.save(newUser);
        return true;
    }

}
```

LoginService.java

```
package com.batch2.artifact1.service;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.batch2.artifact1.domain.Login;
import com.batch2.artifact1.repository.LoginRepo;

@Service
public class LoginService {

    @Autowired
    private LoginRepo rep;

    public Login log(String username, String password) {
        Login user = rep.findByUsernameAndPassword(username, password);
        return user;
    }
}
```

CourseService.java

```
package com.batch2.artifact1.service;

import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.batch2.artifact1.domain.Student;
import com.batch2.artifact1.repository.CourseRepo;

@Service
public class CourseService {
    @Autowired
    private CourseRepo rep;

    public List<Student> listAll() {
        return rep.findAll();
    }

    public void save(Student student) {
        rep.save(student);
    }

    public Student get(long id) {
        return rep.findById(id).orElseThrow(() -> new RuntimeException("Student not found with
ID: " + id));
    }

    public void delete(long id) {
        if (!rep.existsById(id)) {
            throw new RuntimeException("Cannot delete, student not found with ID: " + id);
        }
        rep.deleteById(id);
    }
}
```

RegController.java

```
package com.batch2.artifact1.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.batch2.artifact1.service.RegService;

@Controller
```

```
public class RegController {
    @Autowired
    private RegService service;

    @GetMapping("/register")
    public String showRegisterPage() {
        return "register";
    }

    @PostMapping("/register")
    public String registerUser(@RequestParam String username, @RequestParam String password,
Model model) {
        boolean isRegistered = service.registerUser(username, password);

        if (isRegistered) {
            model.addAttribute("message", "Registration successful! Please login.");
            return "login";
        } else {
            model.addAttribute("error", "Username already exists!");
            return "register";
        }
    }
}
```

LoginController.java

```
package com.batch2.artifact1.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.RequestParam;

import com.batch2.artifact1.domain.Login;
import com.batch2.artifact1.service.LoginService;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;

@Controller
public class LoginController {

    @Autowired
    private LoginService service;

    @GetMapping("/")
    public String showLoginPage() {
        return "login";
    }

    @PostMapping("/login")
    public String processLogin(@RequestParam String username, @RequestParam String password,
Model model) {
```

```
        Login user = service.log(username, password);

        if (user != null) {
            return "redirect:/welcome";
        } else {
            model.addAttribute("error", "Invalid username or password");
            return "login";
        }
    }
}
```

CourseController.java

```
package com.batch2.artifact1.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import com.batch2.artifact1.domain.Student;
import com.batch2.artifact1.service.CourseService;

@Controller
public class CourseController {
    @Autowired
    private CourseService service;

    @GetMapping("welcome")
    public String viewHomePage(Model model) {
        model.addAttribute("liststudent", service.listAll());
        return "welcome";
    }

    @GetMapping("new")
    public String addStudentForm(Model model) {
        model.addAttribute("student", new Student());
        return "new";
    }

    @PostMapping("save")
    public String saveStudent(@ModelAttribute Student student) {
        service.save(student);
        return "redirect:/welcome";
    }

    @GetMapping("edit/{id}")
    public String showEditForm(@PathVariable long id, Model model) {
```

```
        model.addAttribute("student", service.get(id));
        return "new";
    }

    @GetMapping("delete/{id}")
    public String deleteStudent(@PathVariable long id) {
        service.delete(id);
        return "redirect:/welcome";
    }

    @GetMapping("/logout")
    public String logout() {
        return "login";
    }
}
```

Artifact1Application.java

```
package com.batch2.artifact1;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class Artifact1Application {

    public static void main(String[] args) {
        SpringApplication.run(Artifact1Application.class, args);
    }

}
```

Application.properties

```
spring.application.name=artifact1
spring.datasource.url=jdbc:mysql://localhost:3306/Demo
spring.datasource.username=root
spring.datasource.password=Password@123
```

Output

Register

Username:

Password:

Register

Username already exists!

Already have an account? Login

Register

Username:

Password:

Register

Already have an account? Login

Login page

Username:
Password:
Login

Register

Login page

Username:
Password:
Login

Invalid username or password

Register

Student Course Registation

Add new

Student ID	Student Name	Course	Fee	Edit	Delete

Logout

Create New Student

Student Name

Student Name

Course

Course

Fee

0

Save

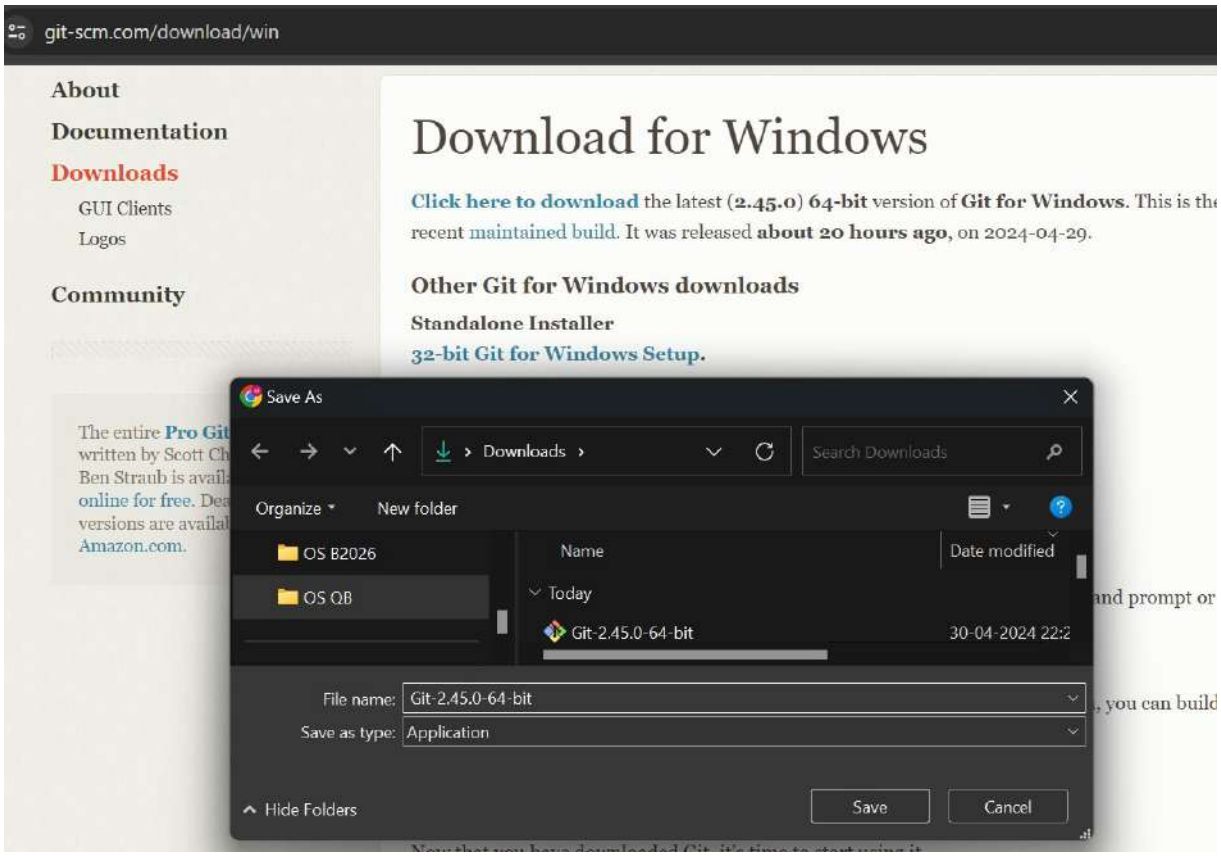
Student Course Registation

Add new

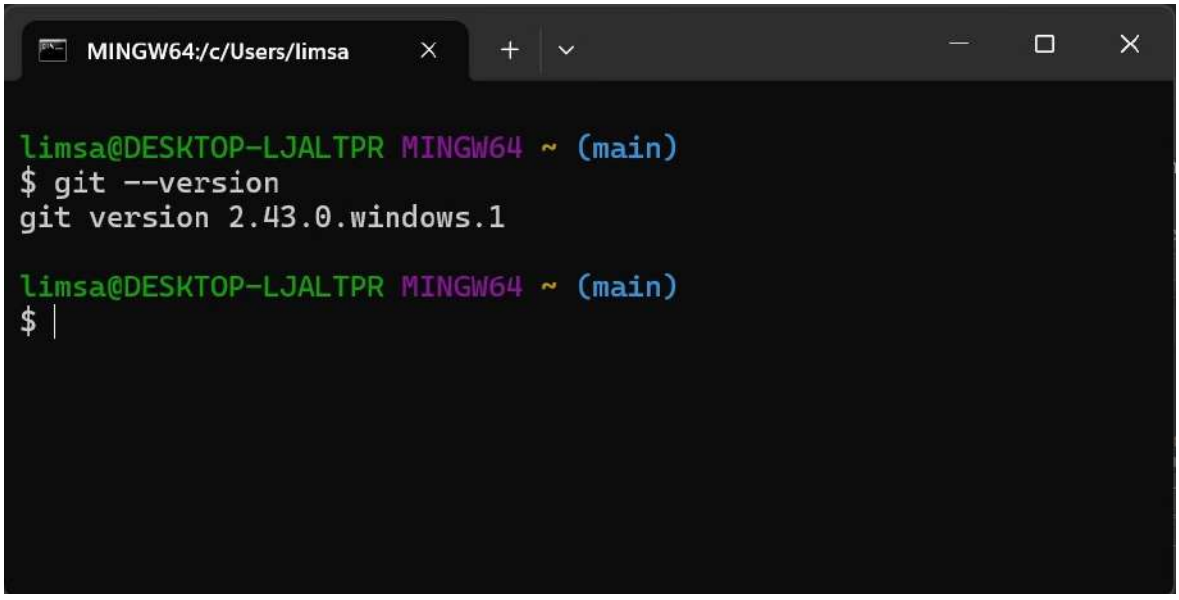
Student ID	Student Name	Course	Fee	Edit	Delete
2	Joshi	Agile Methodologies	25000	Edit	Delete
3	Limsajoshi	Full stack	67000	Edit	Delete
9	Limsa Snowslin	DevOps	50000	Edit	Delete
10	Walter	Java	10000	Edit	Delete
11	New	HTML	5000	Edit	Delete

Ex No: 3 Install Git version control system and configure with GitHub

To install git on windows Go to the Git official website: https://git-scm.com/downloads

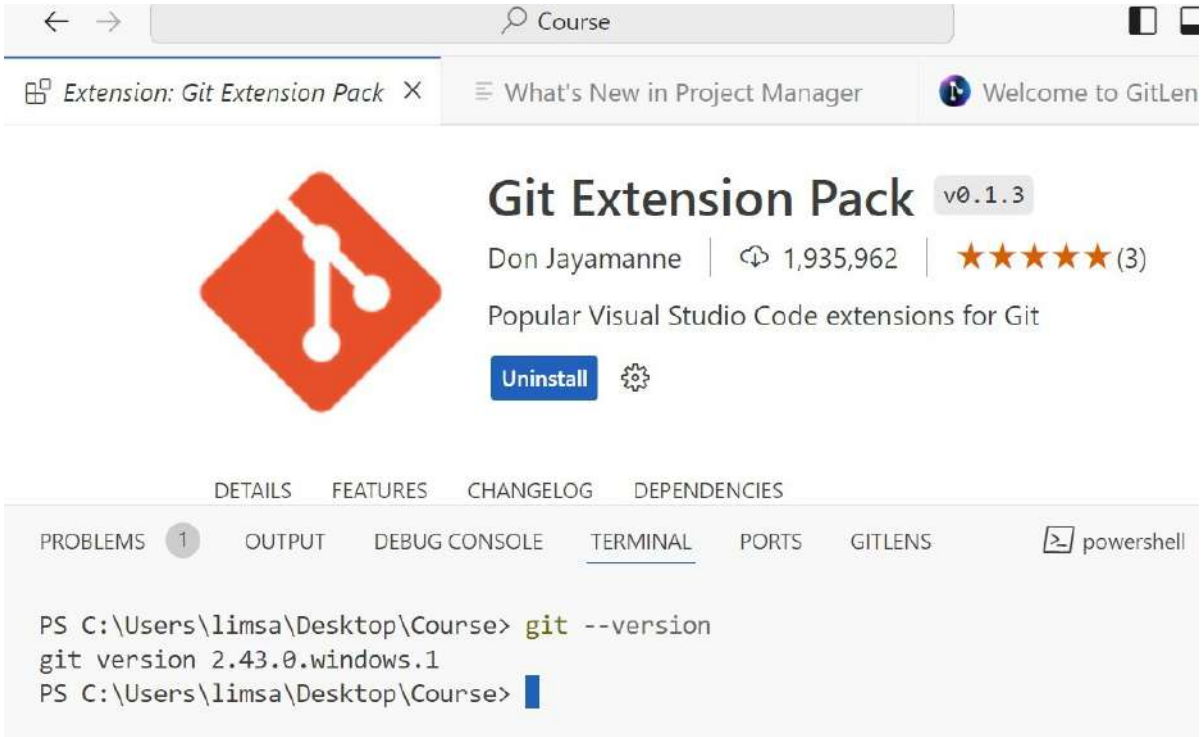


Run the setup and install git



Note: Now you can use git version control for the projects developed in the local machine and connect to the remote repository using git bash/windows cmd prompt or using any IDE.

Git Extension pack for Visual Studio Code



To initialize a new Git repository, navigate to the project directory and run the **command "git init"**

Set the git configuration using the below commands

git config --global --user.name "***"**

git config --global --user.Email "****"**

Set the configuration using the command **"git config --global --list"**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS        powershell

PS C:\Users\limsa\Desktop\Spring\Employee> git init
Initialized empty Git repository in C:/Users/limsa/Desktop/Spring/Employee/.git/
PS C:\Users\limsa\Desktop\Spring\Employee> git config --global --list
user.name=LimsaJoshi
user.email=limsajoshi@gmail.com
core.editor="C:\Users\limsa\AppData\Local\Programs\Microsoft VS Code\bin\code" --wait
PS C:\Users\limsa\Desktop\Spring\Employee>
```

Basic git Commands

git add "file name"

git add .

git commit -m "msg"

git status

git log

git branch -m main

git remote add origin "URL"

git push -m origin "branch name"

git checkout -b "new branch name"

git switch -c "new branch name"

git checkout/switch "branch name"

git pull origin "branch name"

git merge "source branch name"

git tag "tag name"

git tag -a "tag name" -m "message"

git push origin tag name

git merge <source_branch>

Command – “git status”

```
PS C:\Users\limsa\Desktop\Spring\Employee> git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        employee/

nothing added to commit but untracked files present (use "git add" to track)
PS C:\Users\limsa\Desktop\Spring\Employee>
```

The git add command is used in Git to add changes or files to the staging area. This staging area is where you prepare your changes before committing them to the repository. Here's how to use git add: “git add .”

```
PS C:\Users\limsa\Desktop\Spring\Employee> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   employee/.gitignore
        new file:   employee/.mvn/wrapper/maven-wrapper.jar
        new file:   employee/.mvn/wrapper/maven-wrapper.properties
        new file:   employee/mvnw
        new file:   employee/mvnw.cmd
        new file:   employee/pom.xml
        new file:   employee/src/main/java/com/example/employee/Employee.java
        new file:   employee/src/main/java/com/example/employee/EmployeeApplication.java
        new file:   employee/src/main/java/com/example/employee/EmployeeController.java
        new file:   employee/src/main/java/com/example/employee/EmployeeRepository.java
        new file:   employee/src/main/java/com/example/employee/Entity.java
        new file:   employee/src/main/java/com/example/employee/Id.java
        new file:   employee/src/main/resources/application.properties
        new file:   employee/src/main/resources/templates/employees.html
        new file:   employee/src/test/java/com/example/employee/employee/EmployeeApplicationTests.java
```

Once your changes are staged, you can commit them to the Git repository using git commit. The basic syntax for a commit is:

git commit -m "Your commit message here"

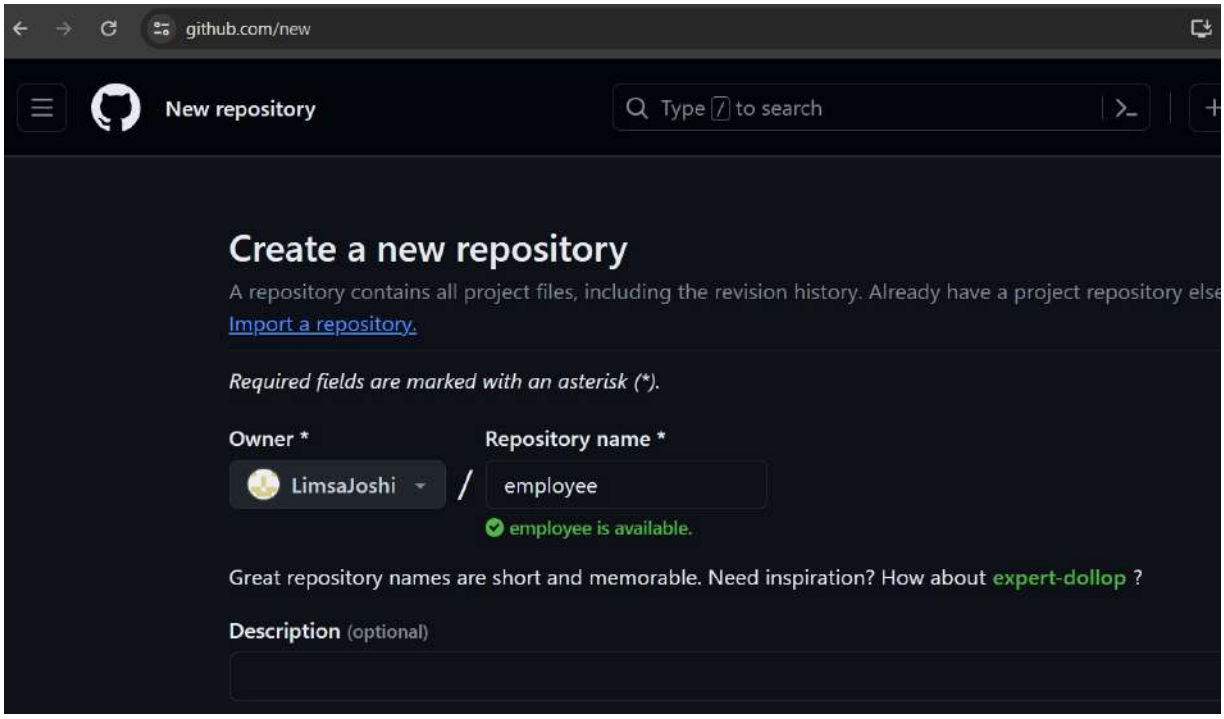
```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS          powershell

PS C:\Users\limsa\Desktop\Spring\Employee> git commit -m "First Commit"
[master (root-commit) 9cae115] First Commit
 15 files changed, 812 insertions(+)
 create mode 100644 employee/.gitignore
 create mode 100644 employee/.mvn/wrapper/maven-wrapper.jar
 create mode 100644 employee/.mvn/wrapper/maven-wrapper.properties
 create mode 100644 employee/mvnw
 create mode 100644 employee/mvnw.cmd
 create mode 100644 employee/pom.xml
 create mode 100644 employee/src/main/java/com/example/employee/Employee.java
 create mode 100644 employee/src/main/java/com/example/employee/EmployeeApplication.java
 create mode 100644 employee/src/main/java/com/example/employee/EmployeeController.java
 create mode 100644 employee/src/main/java/com/example/employee/EmployeeRepository.java
 create mode 100644 employee/src/main/java/com/example/employee/Entity.java
 create mode 100644 employee/src/main/java/com/example/employee/Id.java
 create mode 100644 employee/src/main/resources/application.properties
 create mode 100644 employee/src/main/resources/templates/employees.html
 create mode 100644 employee/src/test/java/com/example/employee/employee/EmployeeApplicationTests.java
PS C:\Users\limsa\Desktop\Spring\Employee>
```

The “**git log**” command is used in Git to view the commit history of a repository. It displays a list of commits along with their commit messages, author details, commit dates, and commit hashes.

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

PS C:\Users\limsa\Desktop\Spring\Employee> git log
commit 9cae11501baef00580dfe9f88d595da763d5849e (HEAD -> master)
Author: LimsaJoshi <limsajoshi@gmail.com>
Date:   Tue Apr 30 22:56:14 2024 +0530

    First Commit
PS C:\Users\limsa\Desktop\Spring\Employee>
```

Log into github and create a new repository

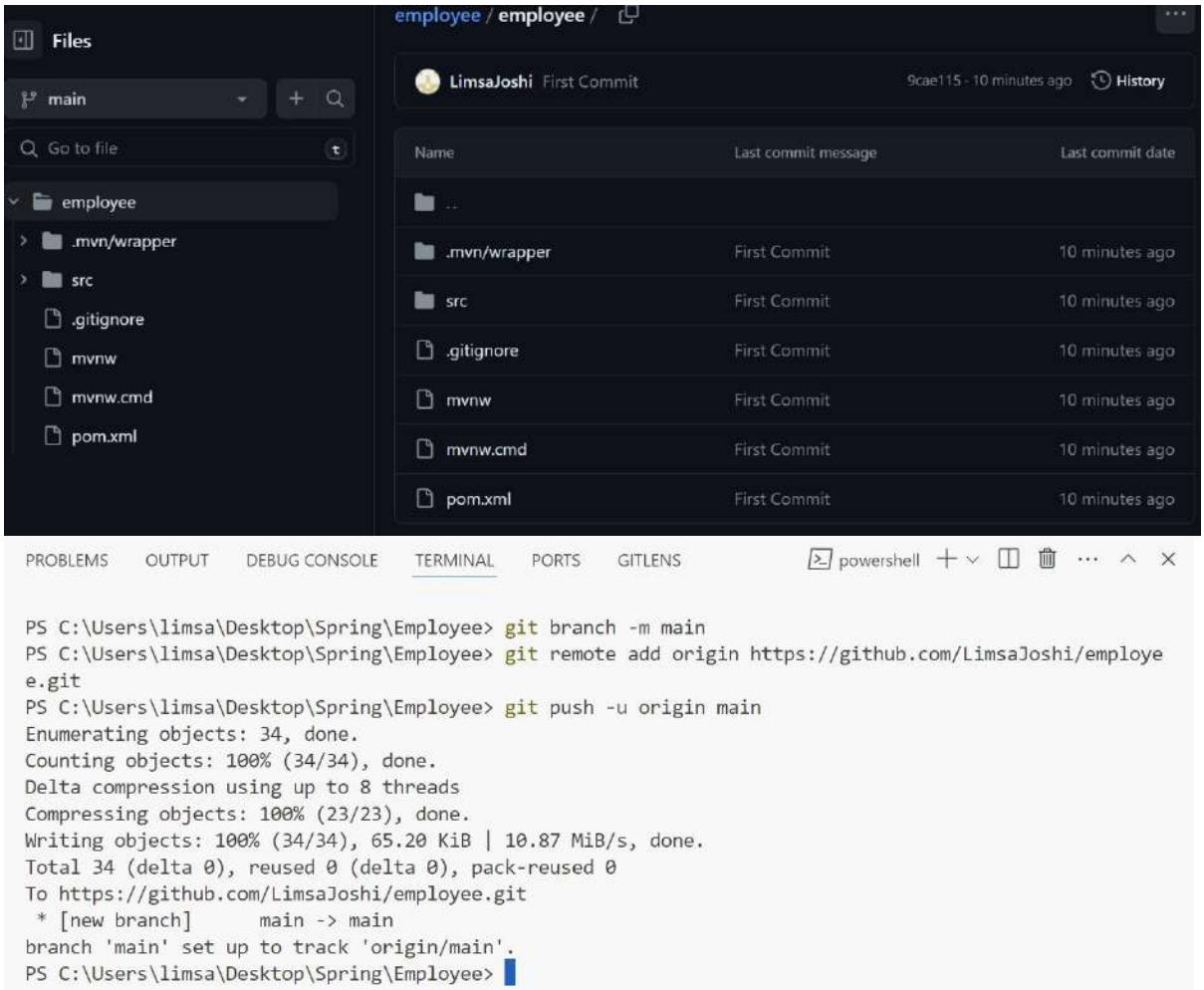
And change the branch master to main “git branch -m main” (optional)



Copy github repository URL

Run the git remote command **“git remote add origin "URL"”** to connect to the github remote repository (origin)

git push -m origin "branch name"



Added Readme.md in the remote repository, to pull the changes to the local machine, run the git pull command

Ex No: 4 Installation and build using Maven

To install Apache Maven on Windows, follow these steps:

Download Apache Maven:

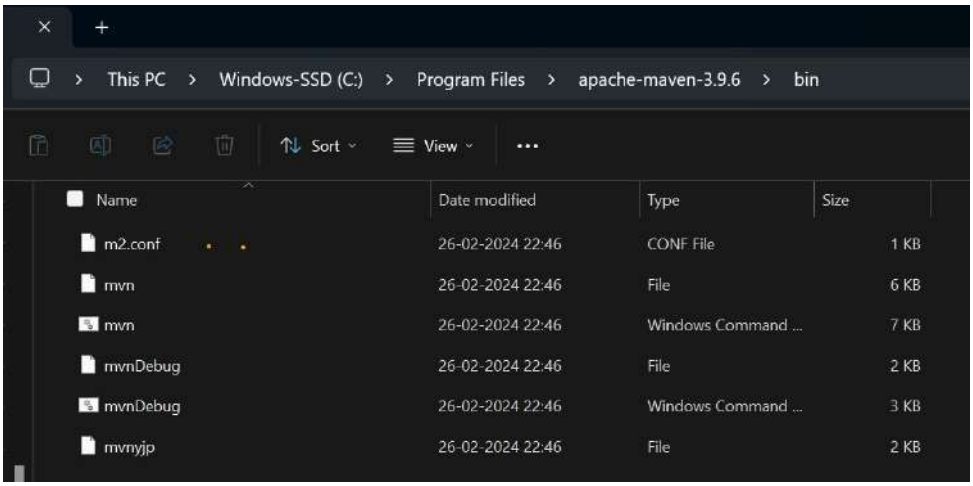
1. Go to the Apache Maven official website: https://maven.apache.org/download.cgi
2. Under "Files," download the latest version of Apache Maven bin in ZIP format.

Extract Maven ZIP File:

1. Once the download is complete, navigate to the directory where the ZIP file was downloaded.
2. Right-click on the ZIP file and select "Extract All" to extract its contents.

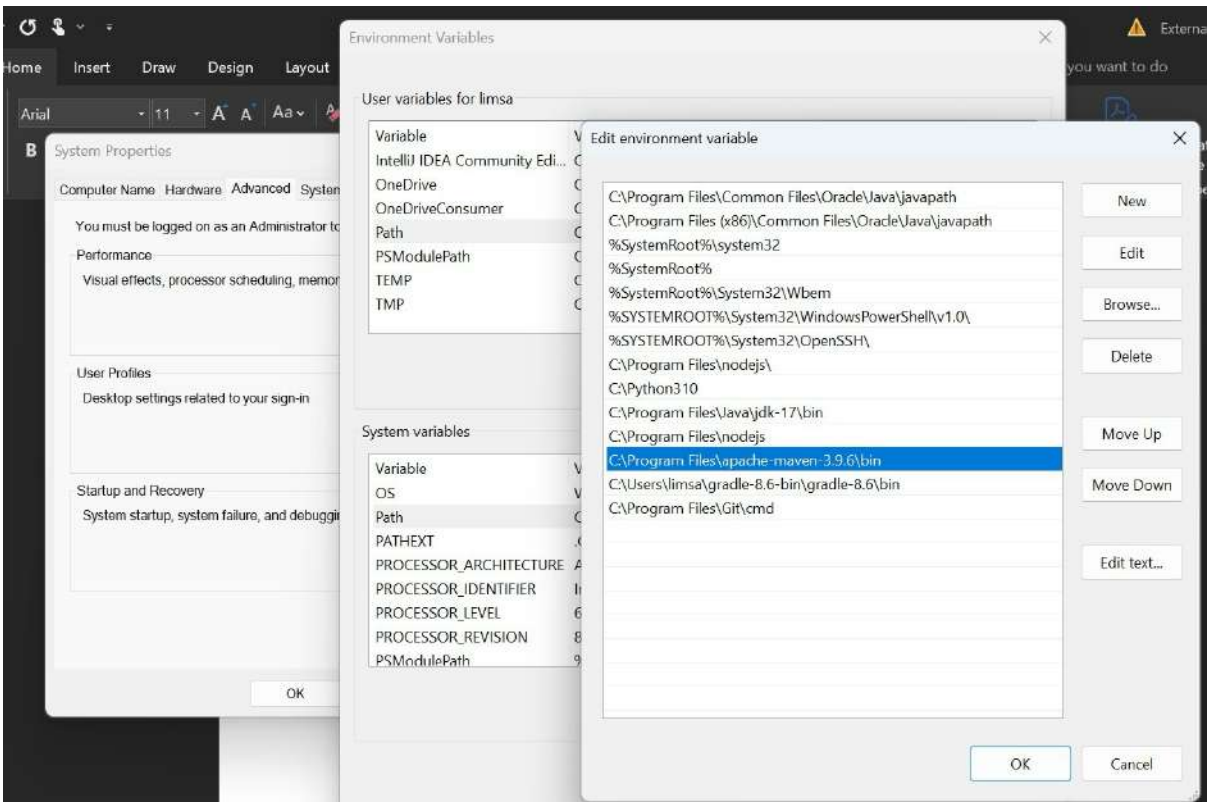
Move Maven Directory:

Move the extracted Apache Maven folder to a location on your computer where you want to install Maven. For example, you can move it to C:\Program Files\Apache Maven.



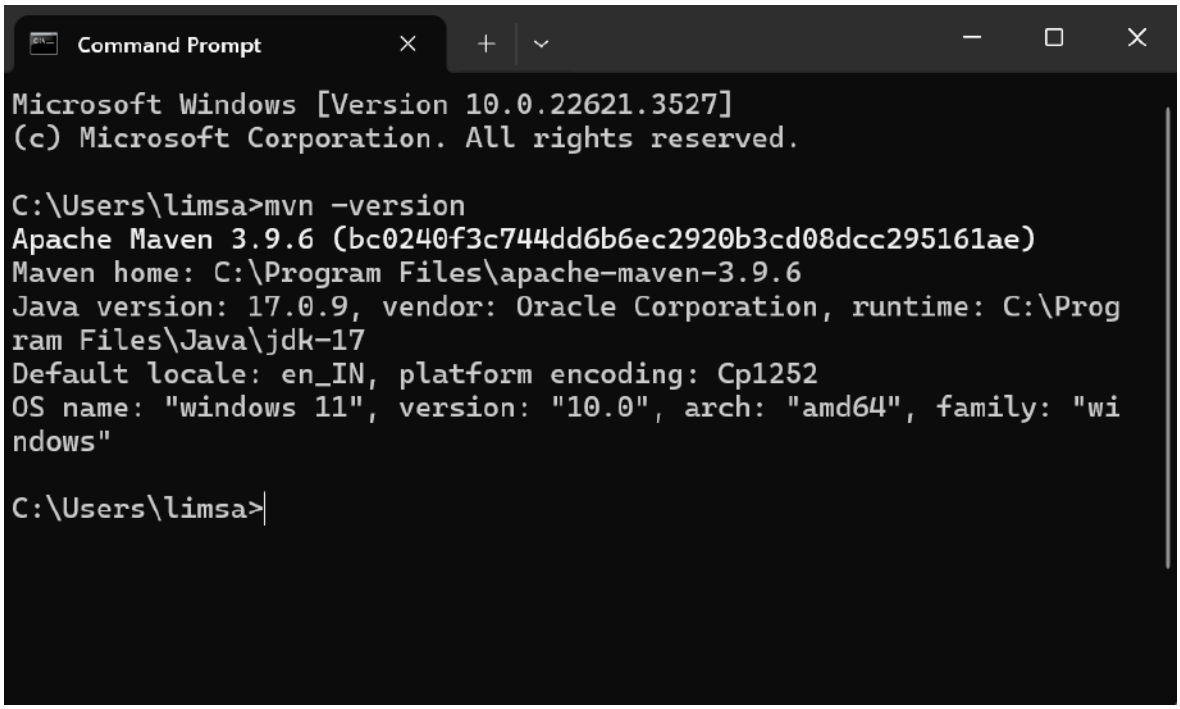
Set Maven Environment Variables:

1. Right-click on "This PC" or "My Computer" and select "Properties."
2. Click on "Advanced system settings" on the left sidebar.
3. In the System Properties window, click on the "Environment Variables" button.
4. In the Environment Variables window, under "System variables", Find the "Path" variable in the list of system variables, select it, and click on "Edit."
5. Click "OK" to save the changes and close all open windows.

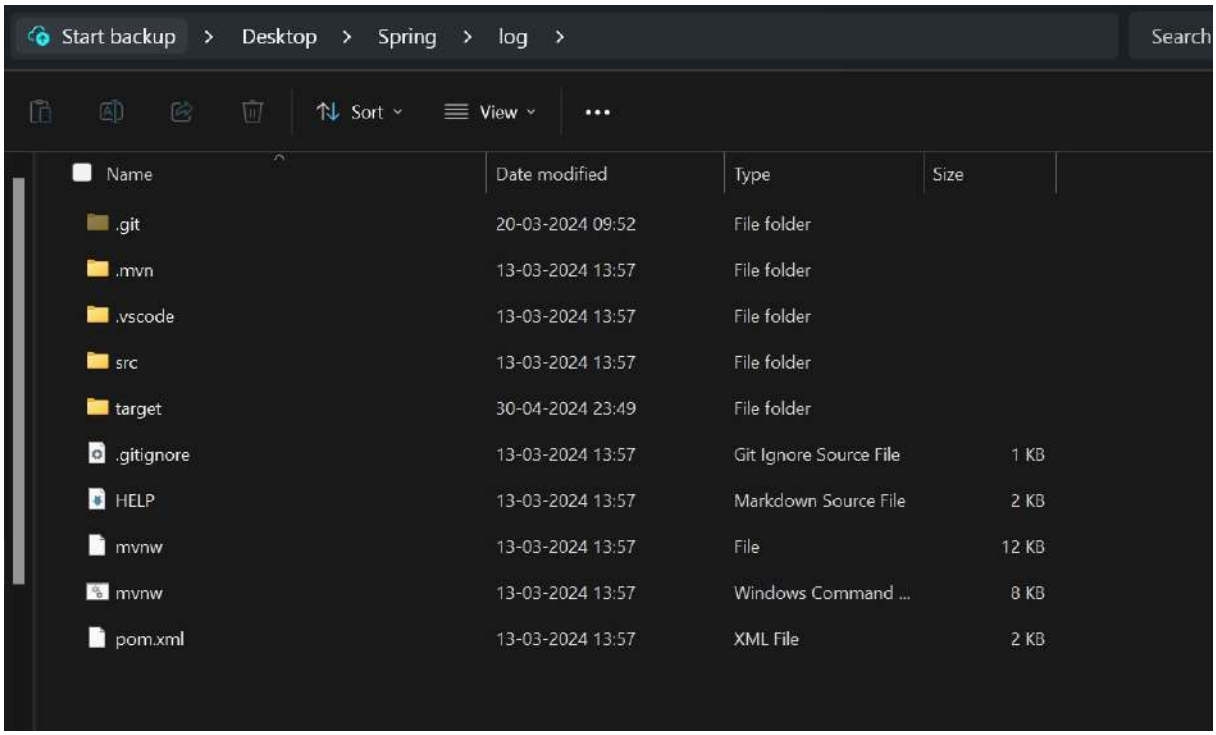


Verify Maven Installation:

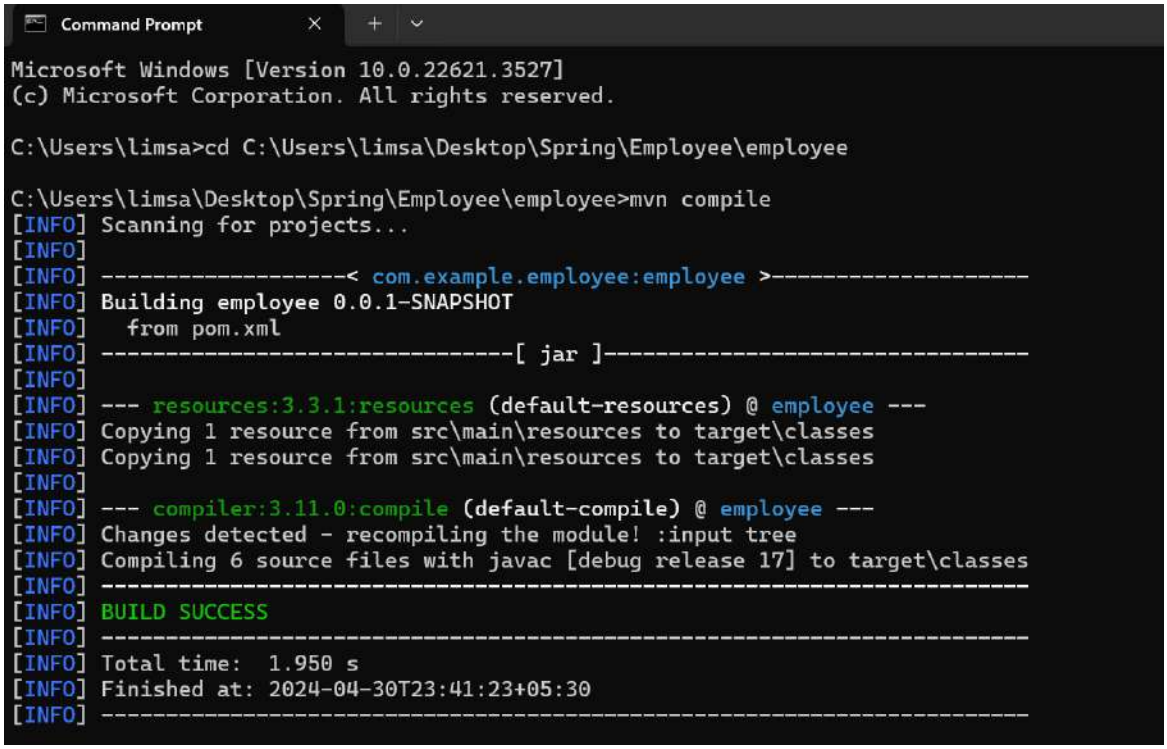
1. Open a new Command Prompt window by searching for "cmd" in the Start menu.
2. Type mvn -version and press Enter.
3. If Maven is installed correctly, you should see the Maven version and other information displayed in the Command Prompt.



Navigate to the project directory where pom.xml is located and run the maven build commands



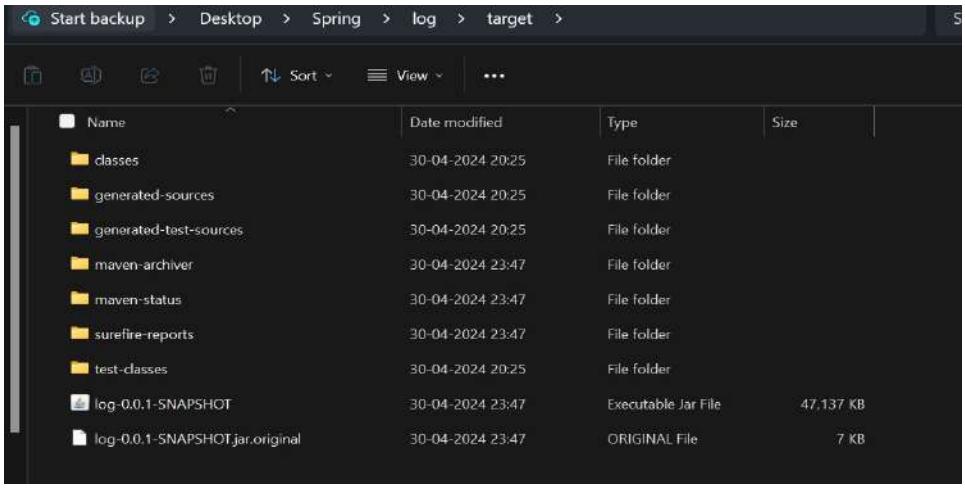
mvn compile

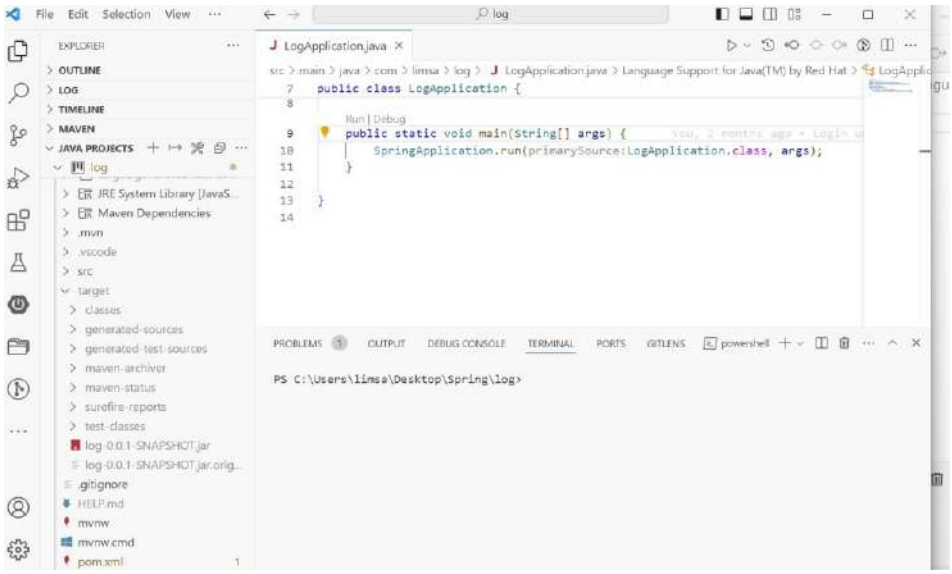


```
Command Prompt
Microsoft Windows [Version 10.0.22621.3527]
(c) Microsoft Corporation. All rights reserved.

C:\Users\limsa>cd C:\Users\limsa\Desktop\Spring\Employee\employee

C:\Users\limsa\Desktop\Spring\Employee\employee>mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -------------------< com.example.employee:employee >--------------------
[INFO] Building employee 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ employee ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ employee ---
[INFO] Changes detected - recompiling the module! :input tree
[INFO] Compiling 6 source files with javac [debug release 17] to target\classes
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  1.950 s
[INFO] Finished at: 2024-04-30T23:41:23+05:30
[INFO] ------------------------------------------------------------------------
```

mvn install

```
Command Prompt
has been appended
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 4.897 s -- in com.limsa.log.LogApplicationTests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- jar:3.3.0:jar (default-jar) @ log ---
[INFO] Building jar: C:\Users\limsa\Desktop\Spring\log\target\log-0.0.1-SNAPSHOT.jar
[INFO]
[INFO] --- spring-boot:3.2.3:repackage (repackage) @ log ---
[INFO] Replacing main artifact C:\Users\limsa\Desktop\Spring\log\target\log-0.0.1-SNAPSHOT.jar with repackaged archive,
adding nested dependencies in BOOT-INF/.
[INFO] The original artifact has been renamed to C:\Users\limsa\Desktop\Spring\log\target\log-0.0.1-SNAPSHOT.jar.origina
l
[INFO]
[INFO] --- install:3.1.1:install (default-install) @ log ---
[INFO] Installing C:\Users\limsa\Desktop\Spring\log\pom.xml to C:\Users\limsa\.m2\repository\com\limsa\log\0.0.1-SNAPSHO
T\log-0.0.1-SNAPSHOT.pom
[INFO] Installing C:\Users\limsa\Desktop\Spring\log\target\log-0.0.1-SNAPSHOT.jar to C:\Users\limsa\.m2\repository\com\l
imsa\log\0.0.1-SNAPSHOT\log-0.0.1-SNAPSHOT.jar
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  10.507 s
[INFO] Finished at: 2024-04-30T23:47:22+05:30
[INFO] ------------------------------------------------------------------------
```





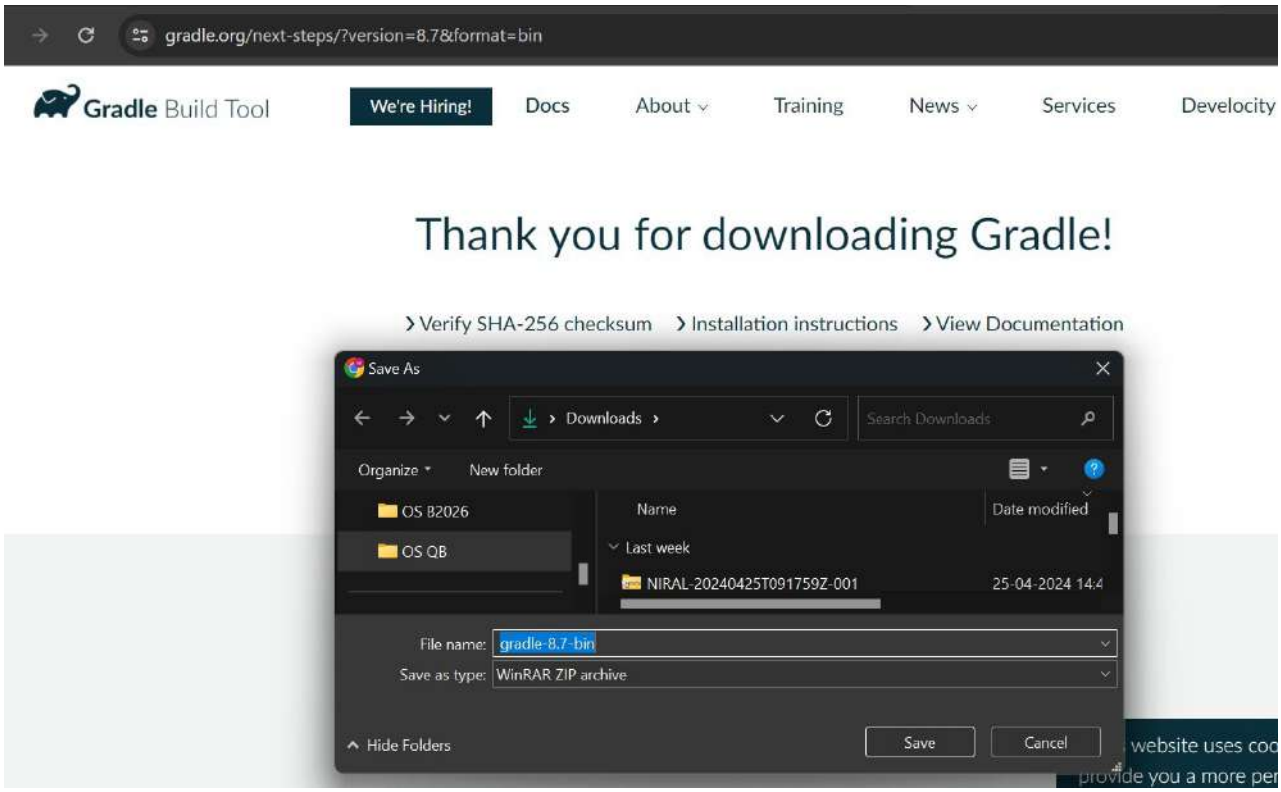
After the build is successful the jar package is created for the project under target directory

Ex No: 5 Installation and build using Gradle

Here are the steps to install Gradle and perform a build on Windows:

Download Gradle:

1. Go to the Gradle official website: https://gradle.org/install/
2. Download the latest version of Gradle from the "Download the latest version" section. Choose the "Complete" distribution ZIP file.

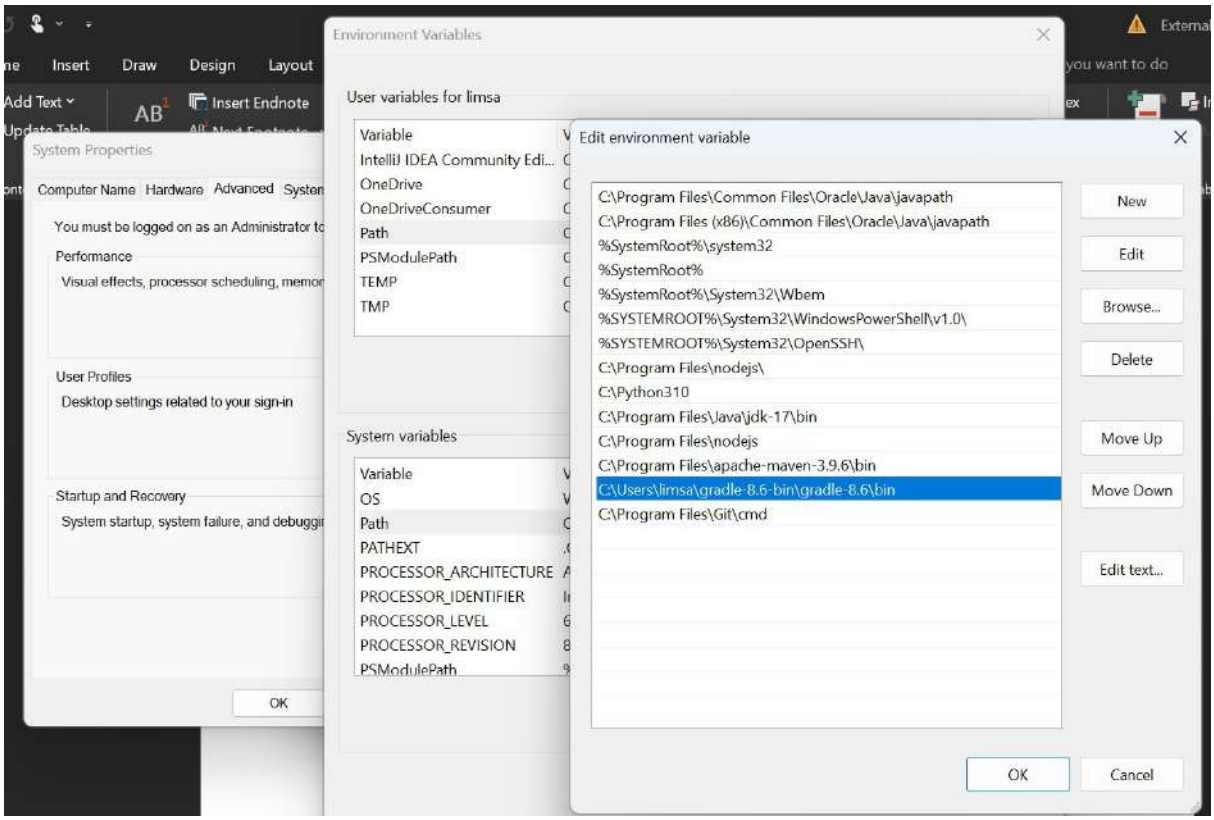


Extract Gradle ZIP File:

1. Once the download is complete, navigate to the directory where the ZIP file was downloaded.
2. Right-click on the ZIP file and select "Extract All" to extract its contents.

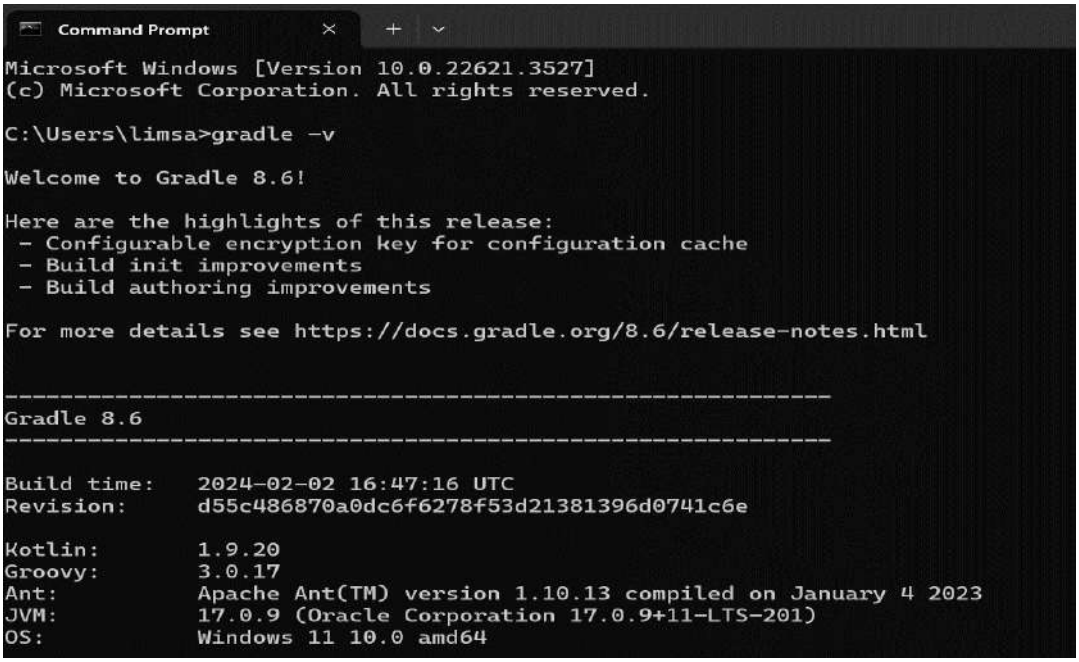
Set Gradle Environment Variables:

1. Right-click on "This PC" or "My Computer" and select "Properties."
2. Click on "Advanced system settings" on the left sidebar.
3. In the System Properties window, click on the "Environment Variables" button.
4. In the Environment Variables window, under "System variables", Find the "Path" variable in the list of system variables, select it, and click on "Edit."
5. Click "OK" to save the changes and close all open windows.



Verify Gradle Installation:

1. Open a new Command Prompt window by searching for "cmd" in the Start menu.
2. Type gradle -v and press Enter.
3. If Gradle is installed correctly, you should see the Gradle version and other information displayed in the Command Prompt.



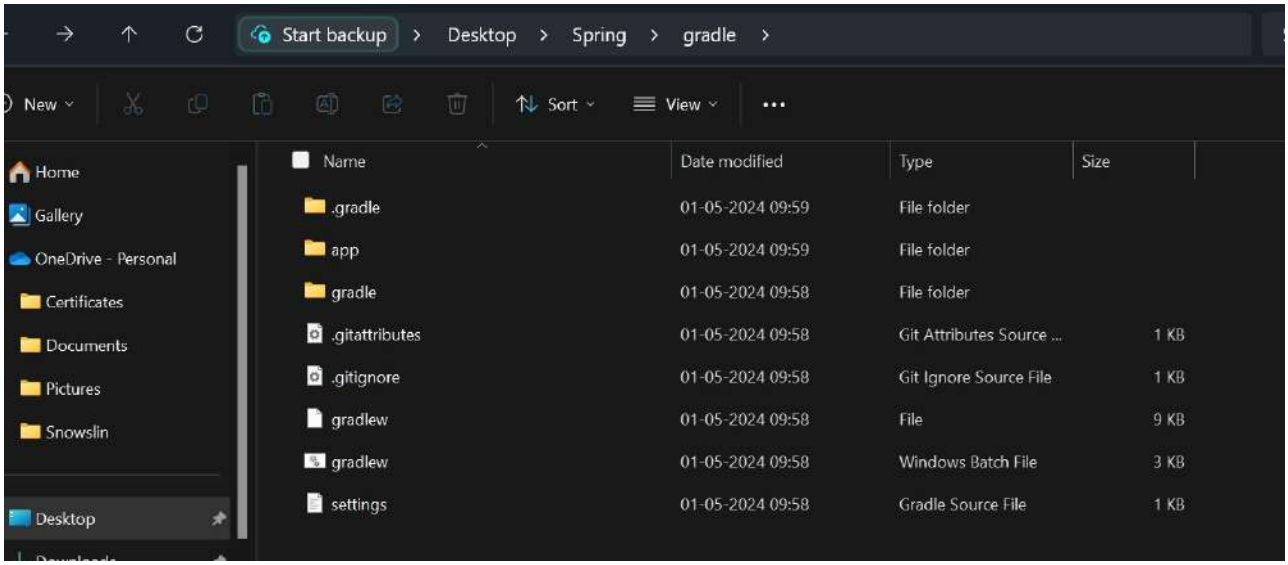
```
Command Prompt
Microsoft Windows [Version 10.0.22621.3527]
(c) Microsoft Corporation. All rights reserved.

C:\Users\limsa>gradle -v

Welcome to Gradle 8.6!

Here are the highlights of this release:
 - Configurable encryption key for configuration cache
 - Build init improvements
 - Build authoring improvements

For more details see https://docs.gradle.org/8.6/release-notes.html


------------------------------------------------------------
Gradle 8.6
------------------------------------------------------------

Build time:   2024-02-02 16:47:16 UTC
Revision:     d55c486870a0dc6f6278f53d21381396d0741c6e

Kotlin:       1.9.20
Groovy:       3.0.17
Ant:          Apache Ant(TM) version 1.10.13 compiled on January 4 2023
JVM:          17.0.9 (Oracle Corporation 17.0.9+11-LTS-201)
OS:           Windows 11 10.0 amd64
```

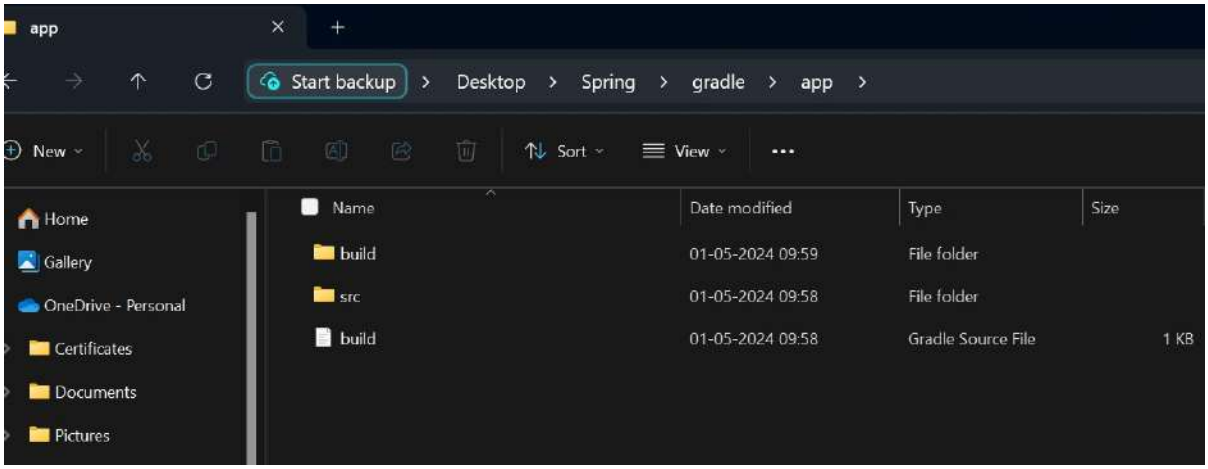
To initialize a Gradle project, you can follow these steps:

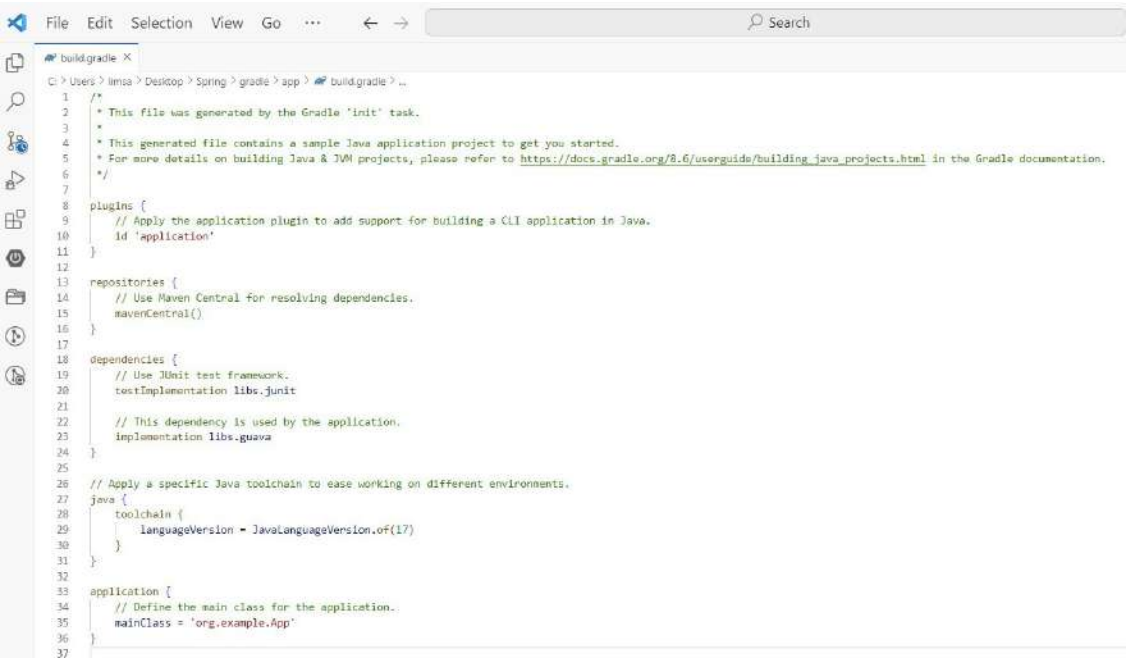
1. Open Terminal or Command Prompt: **Navigate to the directory where you want to create your Gradle project.**
2. Run **"gradle init"** Command
3. **Choose Project Type:** Gradle will prompt you to choose a type for your project:
   - Select **application** if you are creating a Java application.
   - Select library if you are creating a Java library.
   - Select groovy-library if you are creating a Groovy library.
   - Select scala-library if you are creating a Scala library.
4. **Provide Project Name:** Enter a name for your project.
5. **Configure Project:** Gradle will ask you if you want to customize the project configuration. You can choose to customize the configuration or go with the default options.
6. **Build Your Project:** After completing the configuration, Gradle will generate the necessary files and directories for your project.
7. **Open in IDE:** If you want to open the project in an Integrated Development Environment, you can import it using the appropriate option in your IDE.

```
Command Prompt - gradle ir
Microsoft Windows [Version 10.0.22621.3527]
(c) Microsoft Corporation. All rights reserved.

C:\Users\limsa>cd C:\Users\limsa\Desktop\Spring\gradle

C:\Users\limsa\Desktop\Spring\gradle>gradle init

Select type of project to generate:
  1: basic
  2: application
  3: library
  4: Gradle plugin
Enter selection (default: basic) [1..4] 2

Select implementation language:
  1: C++
  2: Groovy
  3: Java
  4: Kotlin
  5: Scala
  6: Swift
Enter selection (default: Java) [1..6] 3

Generate multiple subprojects for application? (default: no) [yes, no] no
```

```
Select build script DSL:
  1: Kotlin
  2: Groovy
Enter selection (default: Kotlin) [1..2] 2

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1

Project name (default: gradle): gradletest
Enter target version of Java (min. 7) (default: 17): 17
Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]
                                                                                                                    no


> Task :init
To learn more about Gradle by exploring our Samples at https://docs.gradle.org/8.6/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 1m 5s
1 actionable task: 1 executed
C:\Users\limsa\Desktop\Spring\gradle>
```


Start backup > Desktop > Spring > gradle >
New
Sort
View
Home
Gallery
OneDrive - Personal
Certificates
Documents
Pictures
Snowslin
Desktop
Name
Date modified
Type
Size
.gradle 01-05-2024 09:59 File folder
app 01-05-2024 09:59 File folder
gradle 01-05-2024 09:58 File folder
.gitattributes 01-05-2024 09:58 Git Attributes Source ... 1 KB
.gitignore 01-05-2024 09:58 Git Ignore Source File 1 KB
gradlew 01-05-2024 09:58 File 9 KB
gradlew 01-05-2024 09:58 Windows Batch File 3 KB
settings 01-05-2024 09:58 Gradle Source File 1 KB


app
Start backup > Desktop > Spring > gradle > app >
New
Sort
View
Home
Gallery
OneDrive - Personal
Certificates
Documents
Pictures
Name
Date modified
Type
Size
build 01-05-2024 09:59 File folder
src 01-05-2024 09:58 File folder
build 01-05-2024 09:58 Gradle Source File 1 KB


File Edit Selection View Go
Search
build.gradle
C: > Users > limsa > Desktop > Spring > gradle > app > build.gradle > ...
/*
* This file was generated by the Gradle 'init' task.
*
* This generated file contains a sample Java application project to get you started.
* For more details on building Java & JVM projects, please refer to https://docs.gradle.org/8.6/userguide/building_java_projects.html in the Gradle documentation.
*/
plugins {
// Apply the application plugin to add support for building a CLI application in Java.
id 'application'
}
repositories {
// Use Maven Central for resolving dependencies.
mavenCentral()
}
dependencies {
// Use JUnit test framework.
testImplementation libs.junit
// This dependency is used by the application.
implementation libs.guava
}
// Apply a specific Java toolchain to ease working on different environments.
java {
toolchain {
languageVersion = JavaLanguageVersion.of(17)
}
}
application {
// Define the main class for the application.
mainClass = 'org.example.App'
}

To run a Gradle project, use the **"gradle run"** command

```
Command Prompt

C:\Users\limsa\Desktop\Spring\gradle>gradle run

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
C:\Users\limsa\Desktop\Spring\gradle>
```

1. To **create a JAR file** using Gradle, you should configure your build.gradle file with the necessary tasks and settings.

2. Open your project's build.gradle file in a text editor or an Integrated Development Environment (IDE) that supports Gradle.

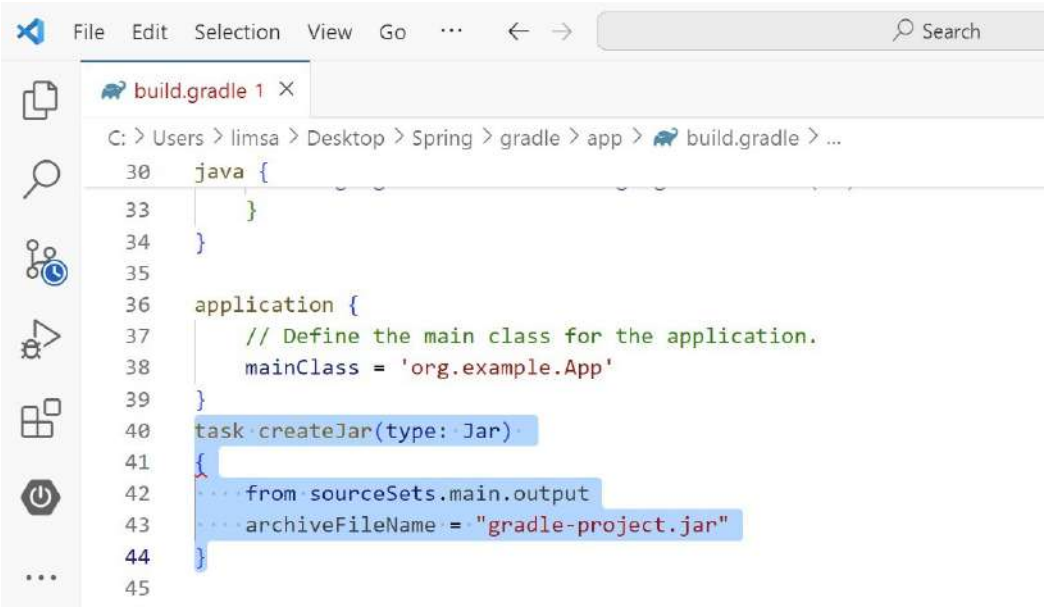
3. **Apply Java Plugin:** Ensure that your build.gradle file applies the Java plugin. If it's not already applied, add the following line at the top of your build.gradle file:

```
plugins
{
id 'java'
}
```

```
build.gradle 1
C: > Users > limsa > Desktop > Spring > gradle > app > build.gradle > plugins
1   /*
2    * This file was generated by the Gradle 'init' task.
3    *
4    * This generated file contains a sample Java application project to get you started.
5    * For more details on building Java & JVM projects, please refer to https://docs.gradle.org/8.6/us
6    */
7
8   plugins {
9       id 'java'
10      // Apply the application plugin to add support for building a CLI application in Java.
11      id 'application'
12
13
14  }
15
```

4. **Define Jar Task:** Add a task to create the JAR file. You can customize this task based on your project's structure and requirements. Here's an example:

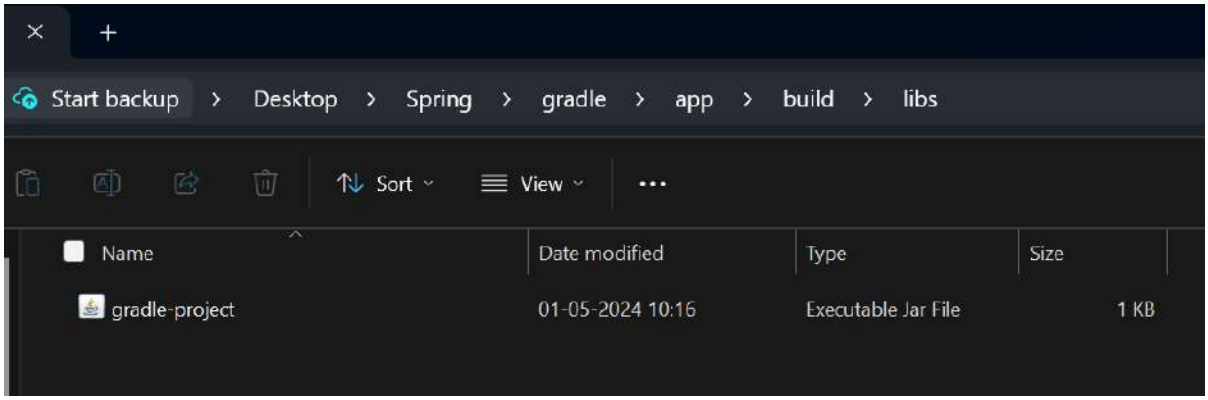
```
task createJar(type: Jar)
{
   from sourceSets.main.output
   archiveFileName = "my-project.jar"
}
```



5. You can now run the createJar task using Gradle using the command **"gradle createJar"**

```
C:\Users\limsa\Desktop\Spring\gradle>gradle createJar

BUILD SUCCESSFUL in 1s
2 actionable tasks: 1 executed, 1 up-to-date
C:\Users\limsa\Desktop\Spring\gradle>
```

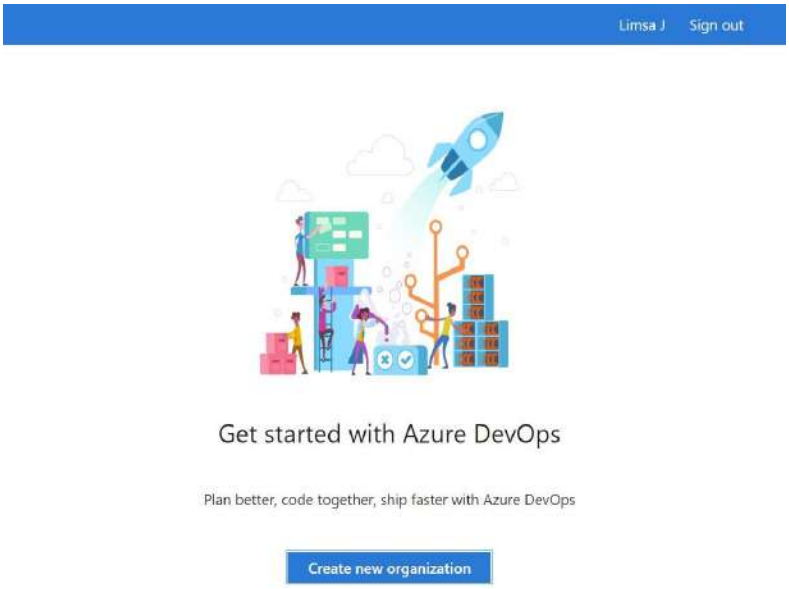
6. **Find the JAR File:** After running the task, the JAR file will be generated in the build/libs directory of your project. You can use this JAR file as needed.



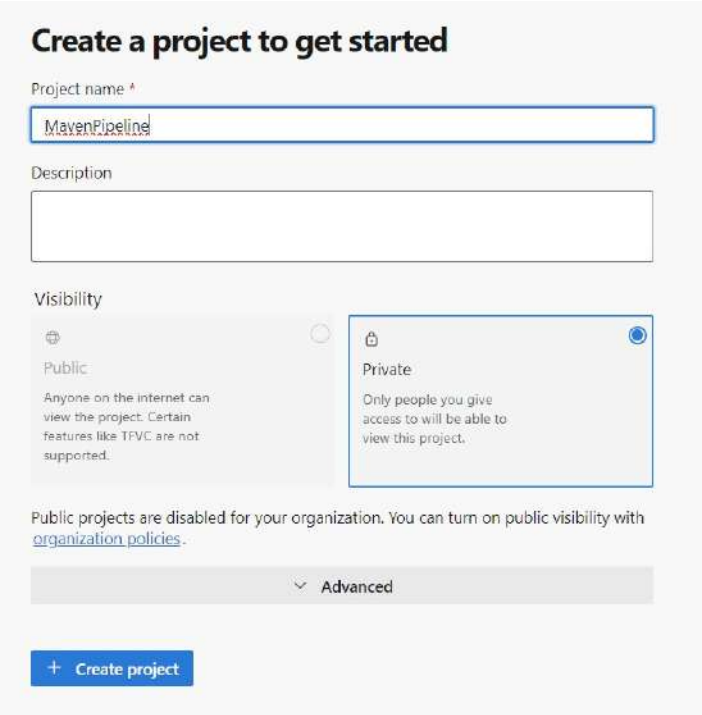
Result:

Ex No. 6 Create a Maven build pipeline in Azure

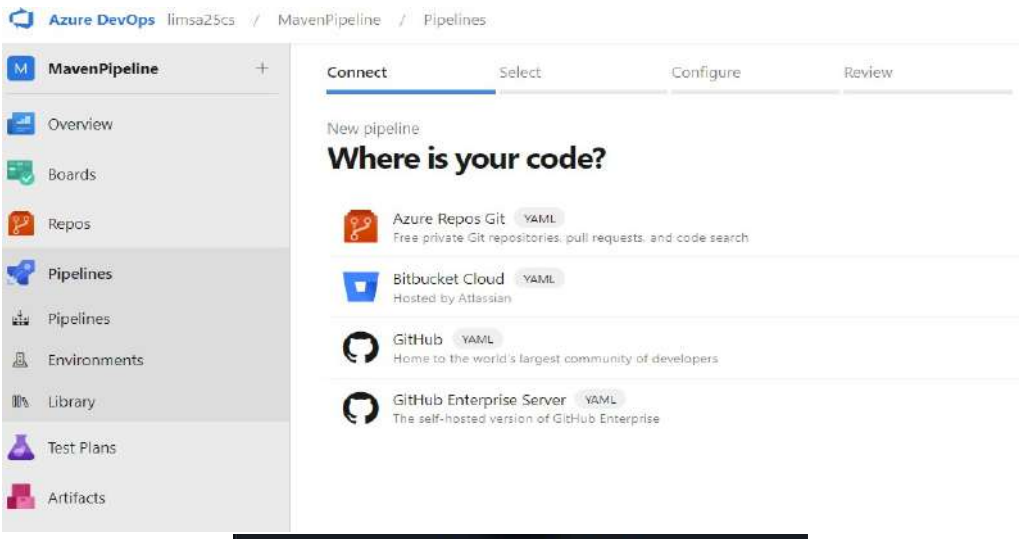
1. **Sign in to Azure:**
   Go to Azure DevOps and create an organization.

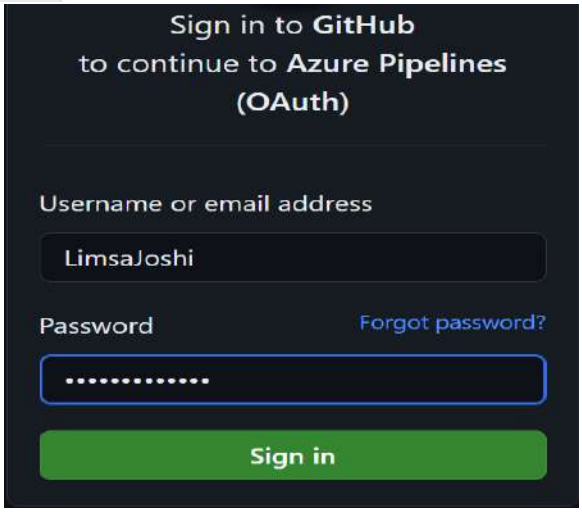


2. **Create a New Project:**
   Create a new one by clicking on "New project" on the Azure DevOps dashboard.



3. **Create Azure Pipeline:**
   Navigate to Pipelines > Pipelines in the Azure DevOps dashboard.
   Click on "New pipeline" to create a new pipeline.





Configure Azure DevOps CI pipeline:

- Use “New pipeleine” in “Pipilenes -> Builds” in Azure DevOps project.
- Connect to GitHub and select your repo. You may want to install Azure DevOps GitHub app and grant access to your GitHub repo.
- Select “Maven” in “Configure your pipeline” step.
- Create new pipeline and push `azure-pipelines.yml` to your repo to keep it alogn with the source code and run build.
- As the result you will get:
    1. CI pipeline configued in Azure DevOps
    2. azure-pipelines.yml config in your repo

Define the Maven goals and options, such as clean, compile, package, test, etc., based on your project's needs.
Ensure you have the necessary plugins and dependencies configured in your pom.xml file.

✓ Connect ✓ Select ✓ Configure Review

New pipeline

Review your pipeline YAML

LimsaJoshi/maven / azure-pipelines.yml

```yaml
# Maven
# Build your Java project and run tests with Apache Maven.
# Add steps that analyze code, save build artifacts, deploy, and more:
# https://docs.microsoft.com/azure/devops/pipelines/languages/java

trigger:
- main

pool: Test-my-computer


steps:
Settings
- task: Maven@4
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'compile'
    publishJUnitResults: true
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
    javaHomeOption: 'JDKVersion'
    mavenVersionOption: 'Default'
    mavenAuthenticateFeed: false
    effectivePomSkip: false
    sonarQubeRunAnalysis: false
```

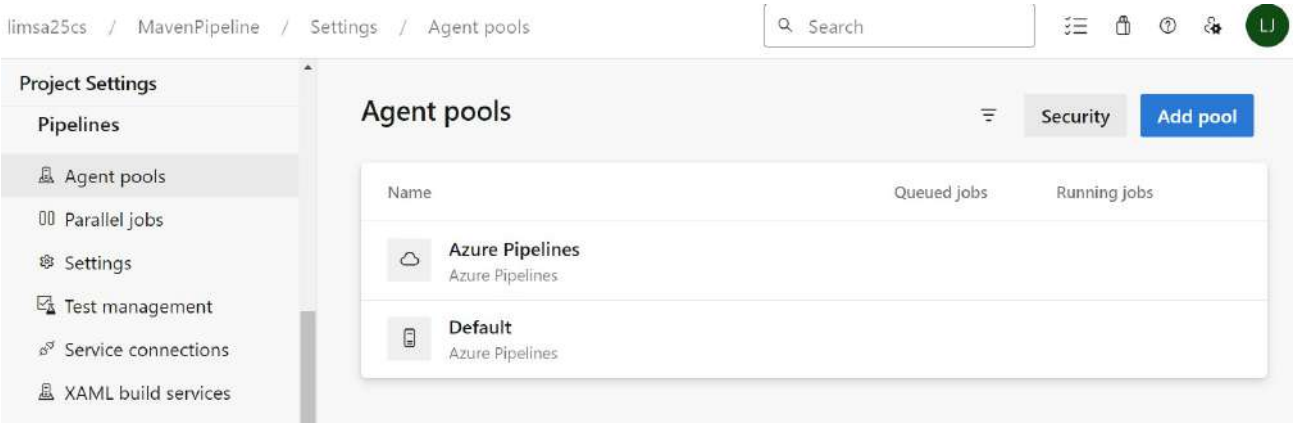
4. **Save and Run Pipeline:**
   Once configured the pipeline, save the changes to the YAML file.
   Optionally, you can run a manual build to test the pipeline's functionality and verify that the Maven build process executes successfully.

5. **Review Build Results:**
   After the pipeline runs, review the build results, logs, and any errors or warnings.
   Azure DevOps provides detailed reports and insights into the build process, including test results if you've configured testing tasks in your pipeline.

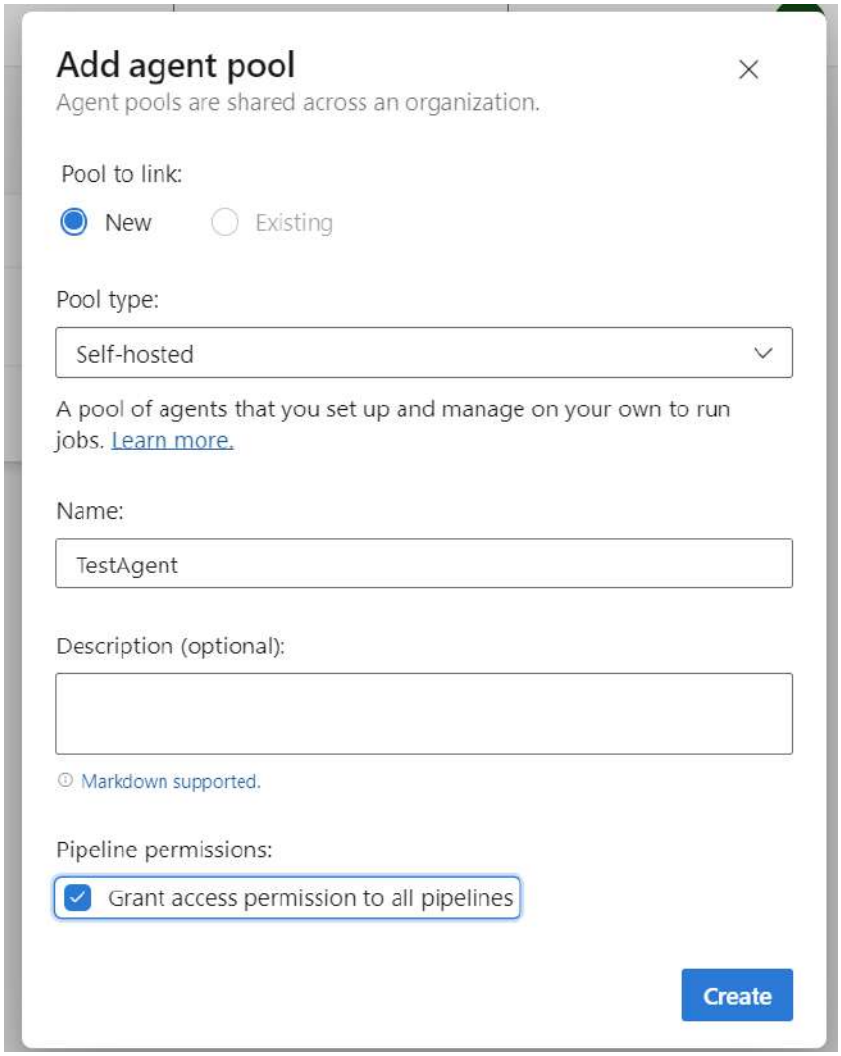
6. **Add Additional Tasks (Optional):**
   Depending on your project's requirements, add additional tasks to the pipeline, such as deploying artifacts to Azure services, running integration tests, or generating build artifacts.
7. **Configure Deployment (Optional):**
   If the pipeline includes deployment tasks, configure the deployment targets and settings accordingly. Deploy Maven artifacts to Azure App Service, Azure Kubernetes Service (AKS), Azure Functions, or any other Azure resources.

Steps to create a Self-Hosted Agent

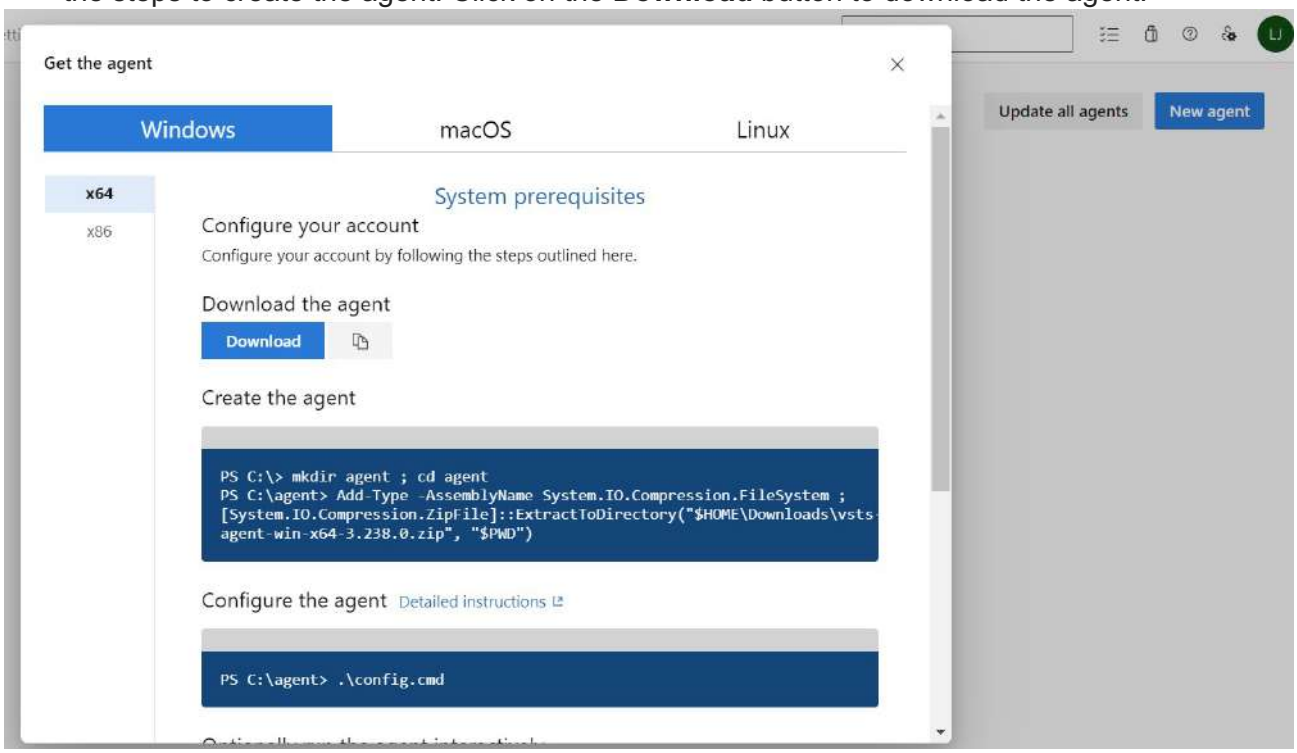
1. Open your web browser and log in to your Azure DevOps account.
2. Navigate to your Azure DevOps project and click on **Project settings** in the left side of the page.
3. Click on **Agent Pools** under Pipelines and click on **Add pool**.



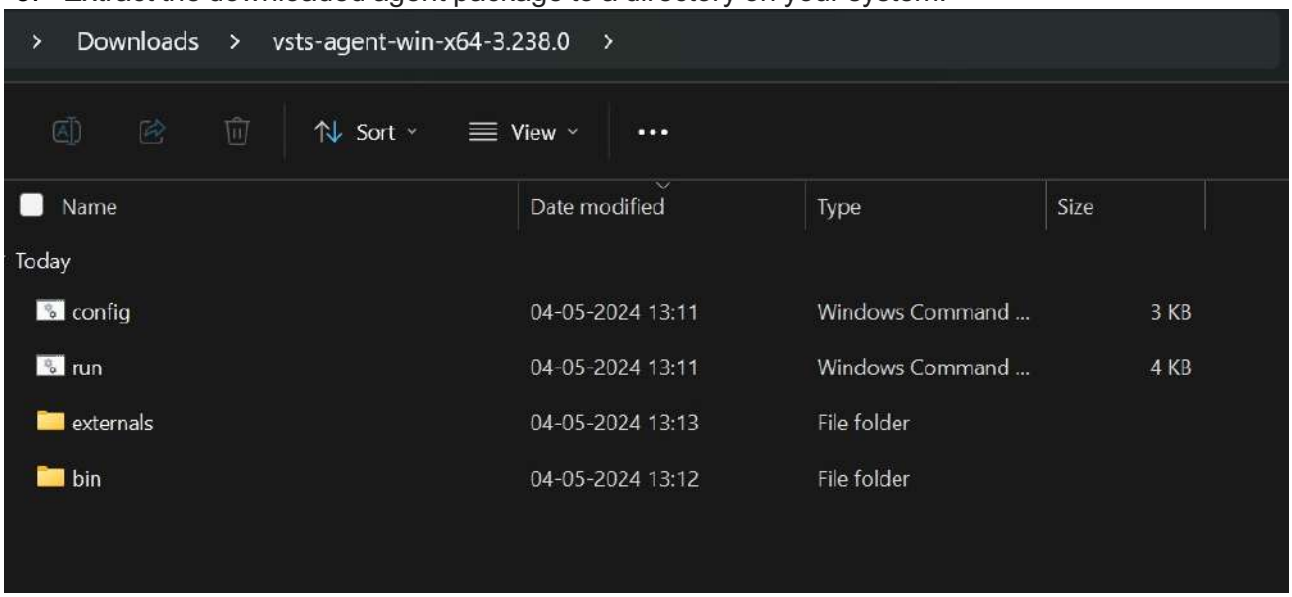
1. Select **Pool-type** as **Self-hosted**, give it a name, grant access to all pipellines and click on **Create.**



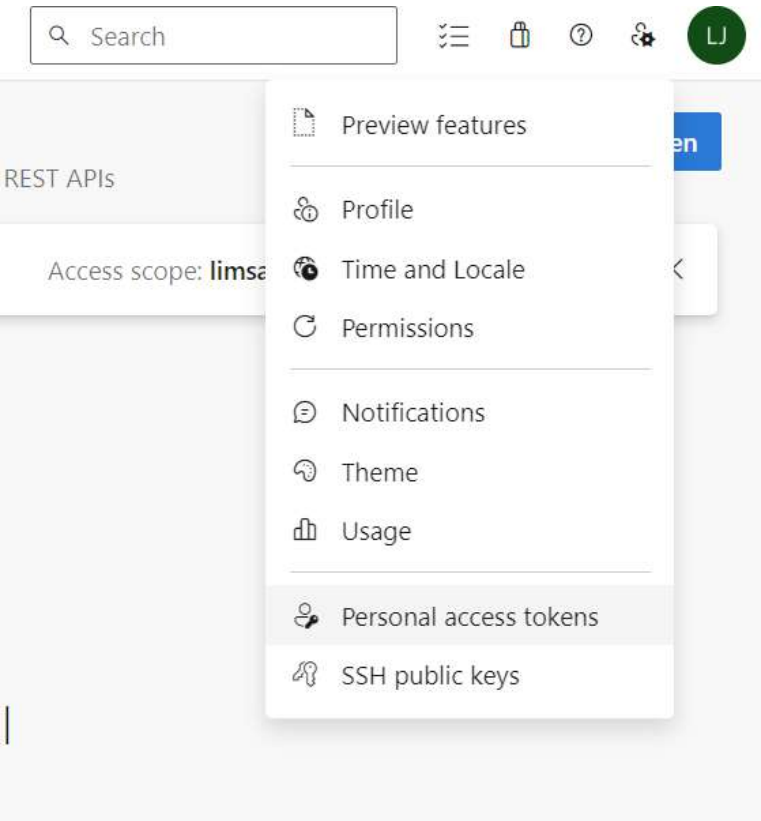
2. Click on the created agent Click on **New agent**. It will show the below screen. Just follow the steps to create the agent. Click on the **Download** button to download the agent.



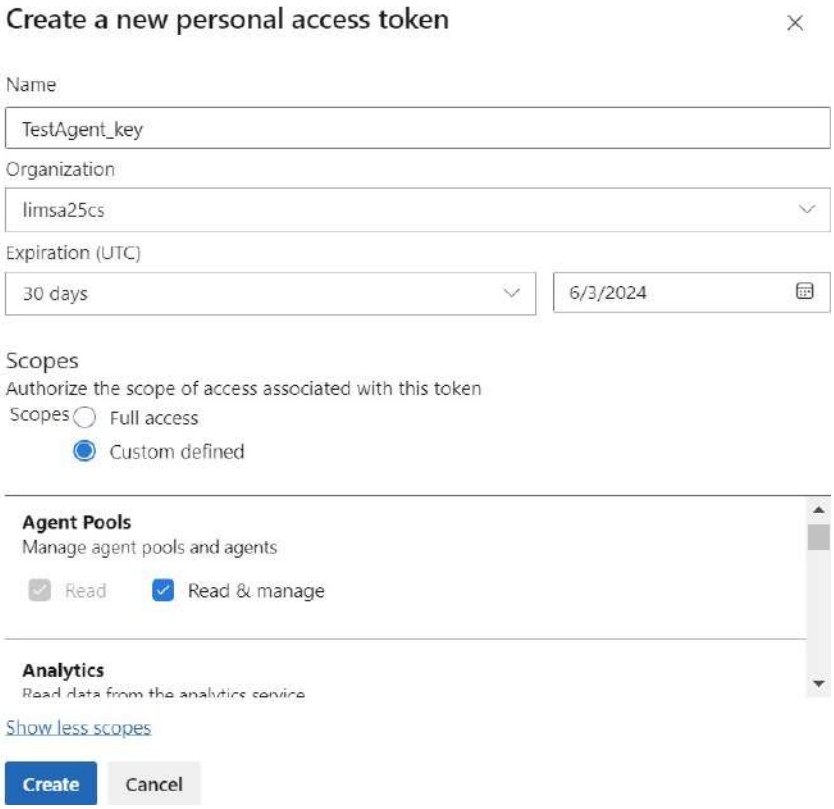
3. Extract the downloaded agent package to a directory on your system.



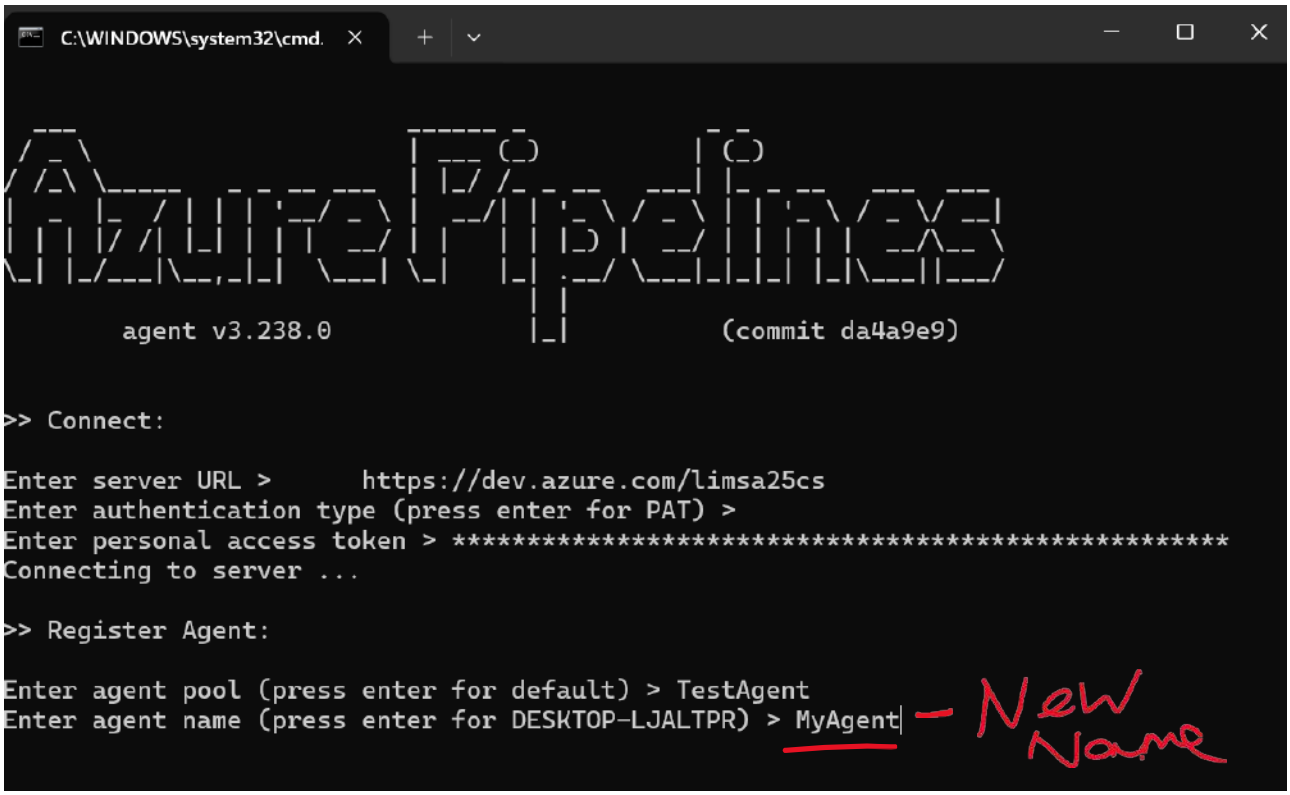
4. Run the configuration script. The configuration script will prompt for your Azure Organization account URL and a personal access token (PAT).
5. To generate a PAT, go to Azure DevOps account, click on the small icon on the left side of the profile picture in the top-right corner, and select **Personal access tokens** from the dropdown menu.



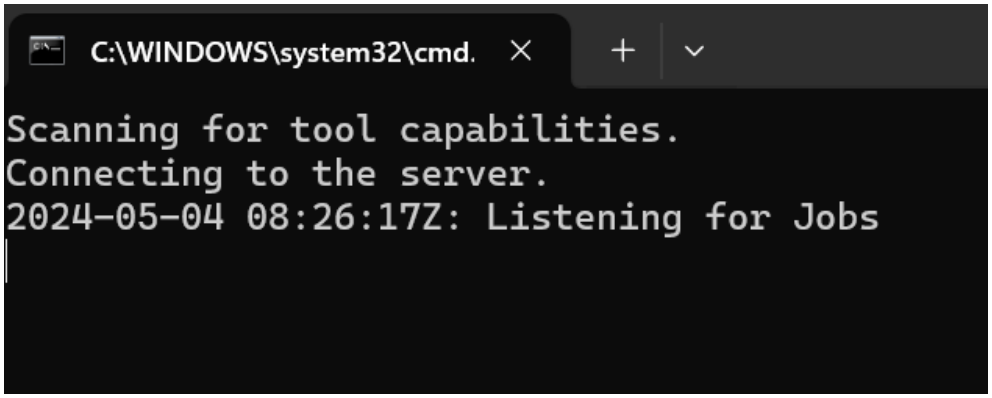
6. Generate a new token with appropriate permissions for the agent.



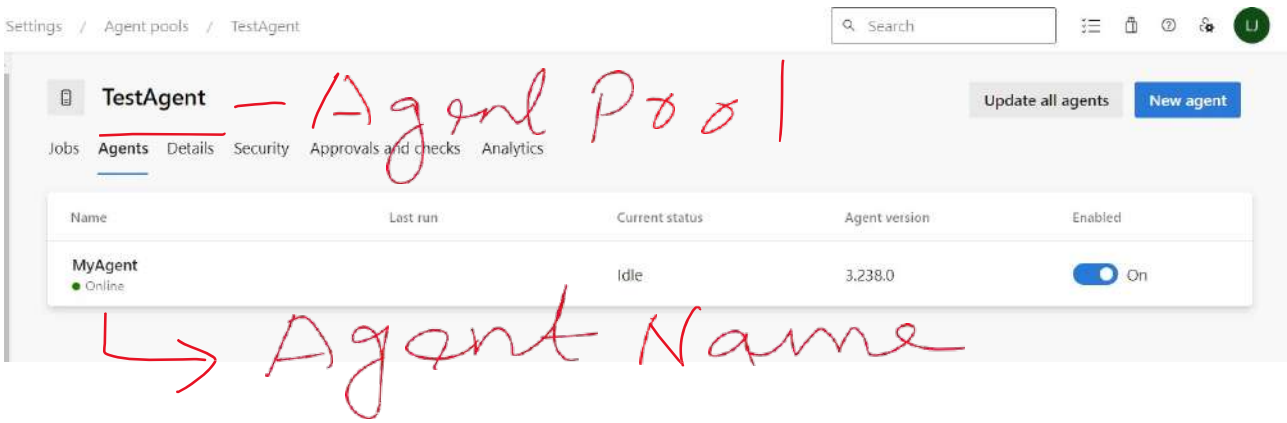
7. Run the configuration file “config.cmd”. Enter the Organization URL, PAT created in the previous step, **Agent pool name created in step 4** and **enter a new name for the agent or press enter for taking the system name as default.**



8. Execute “run.cmd” to run the agent



9. After completing the configuration, the agent will register itself with the specified agent pool in Azure DevOps. Now, go back to agent pool Select the created Agent Pool (TestAgent), you will see the agent (MyAgent) online.

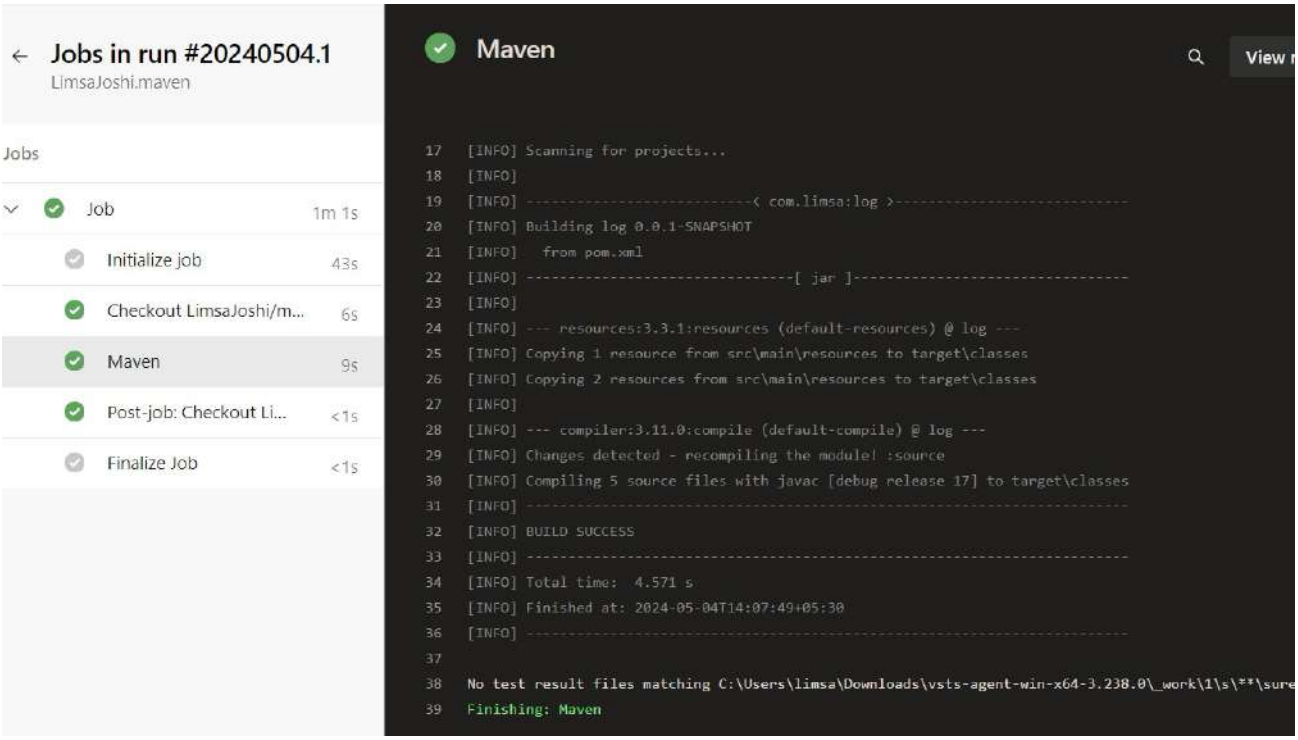


10. Now use the self-hosted agent in the pipeline

← LimsaJoshi.maven

main LimsaJoshi/maven / azure-pipelines.yml

```yaml
# Maven
# Build your Java project and run tests with Apache Maven.
# Add steps that analyze code, save build artifacts, deploy, and more:
# https://docs.microsoft.com/azure/devops/pipelines/languages/java

trigger:
- main

pool: TestAgent


steps:
Settings
- task: Maven@4
  inputs:
    mavenPomFile: 'pom.xml'
    goals: 'compile'
    publishJUnitResults: true
    testResultsFiles: '**/surefire-reports/TEST-*.xml'
    javaHomeOption: 'JDKVersion'
    mavenVersionOption: 'Default'
    mavenAuthenticateFeed: false
    effectivePomSkip: false
    sonarQubeRunAnalysis: false
```


Jobs in run #20240504.1
LimsaJoshi.maven
Jobs
Job 1m 1s
Initialize job 43s
Checkout LimsaJoshi/m... 6s
Maven 9s
Post-job: Checkout Li... <1s
Finalize Job <1s
Maven
View
17 [INFO] Scanning for projects...
18 [INFO]
19 [INFO] ----------< com.limsa:log >----------
20 [INFO] Building log 0.0.1-SNAPSHOT
21 [INFO] from pom.xml
22 [INFO] --------[jar]--------
23 [INFO]
24 [INFO] --- resources:3.3.1:resources (default-resources) @ log ---
25 [INFO] Copying 1 resource from src\main\resources to target\classes
26 [INFO] Copying 2 resources from src\main\resources to target\classes
27 [INFO]
28 [INFO] --- compiler:3.11.0:compile (default-compile) @ log ---
29 [INFO] Changes detected - recompiling the module! :source
30 [INFO] Compiling 5 source files with javac [debug release 17] to target\classes
31 [INFO] ----------
32 [INFO] BUILD SUCCESS
33 [INFO] ----------
34 [INFO] Total time: 4.571 s
35 [INFO] Finished at: 2024-05-04T14:07:49+05:30
36 [INFO] ----------
37
38 No test result files matching C:\Users\limsa\Downloads\vsts-agent-win-x64-3.238.0_work\1\s**\sure
39 Finishing: Maven

Ex No. 7 Run regression test using Maven build pipeline in Azure

1. Create an Azure DevOps Pipeline

You need a YAML-based pipeline in Azure DevOps.

1. Go to **Azure DevOps Portal**.
2. Navigate to your **Project** → Click on **Pipelines** → Select **New Pipeline**.
3. Choose your source repository (Azure Repos, GitHub, Bitbucket, etc.).
4. Select **"Pipeline Type"** and edit the azure-pipelines.yml file.

2. Define Your azure-pipelines.yml File

Modify the pipeline YAML to include Maven build and regression tests.

```
trigger:
  - main  # Adjust as per your branch strategy

pool:
  vmImage: 'ubuntu-latest'  # You can use self hosted agent

steps:
  - task: Maven@3
    displayName: 'Maven Build and Regression Test'
    inputs:
      mavenPomFile: 'pom.xml'  # Ensure the correct path to your pom.xml
      goals: 'clean test'
      publishJUnitResults: true  # Publish JUnit test results
      testResultsFiles: '**/target/surefire-reports/TEST-*.xml'
```

3. Configure Your pom.xml for Regression Testing

Ensure your pom.xml includes a test framework like **JUnit or TestNG** and the necessary dependencies.

JUnit Example in pom.xml

```
<dependencies>
    <!-- JUnit Dependency -->
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.13.2</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<dependency>
    <groupId>org.mockito</groupId>
    <artifactId>mockito-core</artifactId>
    <version>5.5.0</version>
    <scope>test</scope>
</dependency>

<build>
    <plugins>
        <plugin>
```

```
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.1.2</version>
      </plugin>
    </plugins>
</build>
```

4. Commit and Push the YAML

Save and commit the azure-pipelines.yml file to your repository. This will trigger the pipeline execution.

5. Monitor Pipeline Execution

1. Navigate to **Azure DevOps** → **Pipelines**.
2. Select your pipeline run to check the status.
3. View logs and ensure regression tests pass.

Sample code for Login Form and JUnit Test Case for Login Form

login.html

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
 <head>
   <title>Login page</title>
 </head>
 <body>
   <h1>Login page</h1>
    <form th:action="@{/login}" method="post">
     <label for="username">Username</label>:
     <input type="text" id="username" name="username" autofocus="autofocus" /> <br />
     <label for="password">Password</label>:
     <input type="password" id="password" name="password" /> <br />
     <input type="submit" value="Login" />
   </form>
   <p th:if="${error}" th:text="${error}" style="color: red;"></p>
       <div align="left">
       <h3><a th:href="@{/register}">Register</a></h3>
   </div>
     </body>
</html>
```

```
Create database Demo;

Use Demo

CREATE TABLE login
(

  username VARCHAR(50),
  password VARCHAR(50)
);

Select * from login

INSERT INTO login (username, password)
VALUES ('limsa', 'limsa@123');
```

Login.java

```
@Entity
@Table(name = "login")
public class Login {
  @Id
  public String username;
  public String password;

  public Login() {
  }

  public Login(String username, String password) {
     this.username = username;
     this.password = password;
  }

  public String getUsername() {
     return username;
  }

  public void setUsername(String username) {
     this.username = username;
  }

  public String getPassword() {
     return password;
```

```
  }

  public void setPassword(String password) {
      this.password = password;
  }
}
```

LoginRepo.java

```
@Repository
public interface LoginRepo extends JpaRepository<Login, String> {
  Login findByUsernameAndPassword(String username, String password);
}
```

LoginService.java

```
@Service
public class LoginService {

  @Autowired
  private LoginRepo rep;

  public Login log(String username, String password) {
      Login user = rep.findByUsernameAndPassword(username, password);
      return user;
  }
}
```

LoginController.java

```
@Controller
public class LoginController {

  @Autowired
  private LoginService service;

  @GetMapping("/")
  public String showLoginPage() {
      return "login";
  }
```

```
  @PostMapping("/login")
  public String processLogin(@RequestParam String username,
                             @RequestParam String password,
                             Model model) {
      Login user = service.log(username, password);
      if (user != null) {
          return "redirect:/welcome";
      } else {
          model.addAttribute("error", "Invalid username or password");
          return "login";
      }
  }
  @GetMapping("/welcome")
  public String showWelcomePage() {
      return "welcome";
  }
```

application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/Demo
spring.datasource.username=root
spring.datasource.password=Password@123
```

welcome.html

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
    xmlns:th="http://www.thymeleaf.org">

<head>
  <link rel="stylesheet"
href="https://stackpath.bootstrapcdn.com/bootswatch/4.5.2/cosmo/bootstrap.min.css" />
  <script src= "https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"
></script>
</head>
<body>
<div>
  <h2 >Student Course Registation</h2>
  <tr>
      <div align = "left" >
```

```
            <h3><a  th:href="@{'/new'}">Add new</a></h3>
        </div>

    </tr>
    <tr>

    <div class="col-sm-5" align = "center">
                <div class="panel-body" align = "center" >
                <table class="table">
  <thead class="thead-dark">
    <tr>
          <th>Student ID</th>
          <th>Student Name</th>
          <th>Course</th>
          <th>Fee</th>
          <th>Edit</th>
          <th>Delete</th>
    </tr>
  </thead>
  <tbody>
      <tr  th:each="student : ${liststudent}">
        <td th:text="${student.id}">Student ID</td>
        <td th:text="${student.studentname}">StudentName</td>
        <td th:text="${student.course}">Course</td>
        <td th:text="${student.fee}">Fee</td>
        <td>
            <a th:href="@{'/edit/' + ${student.id}}">Edit</a>
        </td>
        <td>
            <a th:href="@{'/delete/' + ${student.id}}">Delete</a>
        </td>
        </tr>

  </tbody>
 </table>

                </div>

            </div>
```

```
  </tr>

  </tbody>
  </table>
 <!--<a th:href="@{/logout}">Logout</a>-->
 <form th:action="@{/logout}" method="get">
  <button type="submit">Logout</button>
</form>
</body>
</html>
```

LoginControllerTest.java

```
import static org.mockito.Mockito.when;
import static org.mockito.ArgumentMatchers.anyString;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.post;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@SpringBootTest
@AutoConfigureMockMvc
public class LoginControllerTest {

  @Autowired
  private MockMvc mockMvc;

  @MockBean
  private LoginService loginService;

  @Test
  public void testSuccessfulLogin() throws Exception {
     Login user = new Login();
     user.setUsername("admin");
     when(loginService.log(anyString(), anyString())).thenReturn(user);
     mockMvc.perform(post("/login")
            .param("username", "admin")
            .param("password", "password"))
            .andExpect(status().is3xxRedirection())
            .andExpect(redirectedUrl("/welcome"));
```

```
  }

  @Test
  public void testFailedLogin() throws Exception {
     when(loginService.log(anyString(), anyString())).thenReturn(null);
     mockMvc.perform(post("/login")
            .param("username", "invalidUser")
            .param("password", "wrongPass"))
            .andExpect(status().isOk())
            .andExpect(view().name("login"))
            .andExpect(model().attributeExists("error"));
  }
}
```

Output:

Run 45 - JUnit_TestResults_124 3 results

Run summary | Test results | Filter

Create bug | Update analysis

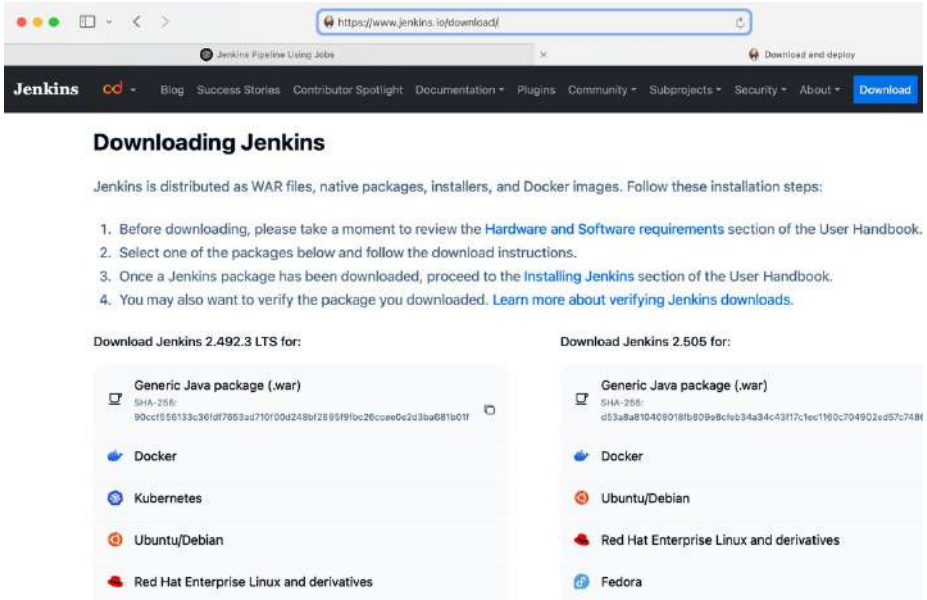
Outcome	Test Case Title	Priority	Duration	Owne
Passed	testSuccessfulLogin	0	0:00:00.017	
Passed	testFailedLogin	0	0:00:00.793	
Failed	contextLoads	0	0:00:00.010	

Ex. No. 8 Jenkins Installation and Setup

Using jenkins.war on localhost

1. Download the Jenkins WAR File

To start the Jenkins installation, download the “jenkins.war” file from the official Jenkins website [jenkins.io] https://www.jenkins.io/download/



2. Run Jenkins Using the WAR File – Prerequisite, Java 17 or 21

Open a terminal or command prompt and navigate to the directory where the “jenkins.war” file is located. Execute the following command to run Jenkins:

```
java -jar jenkins.war
```

This command will start Jenkins on your localhost at port 8080.

3. Access Jenkins

Once Jenkins is up and running, open your web browser and go to “localhost:portnumber”. You will see the Jenkins setup screen.

If the default port (8080) is busy run Jenkins using the following command:

Java -jar Jenkins.war - -httpPort=9000

4. Unlock Jenkins

You will be prompted to unlock Jenkins using an automatically generated password. Find the initialAdminPassword file in your Jenkins home directory:

cat ~/.jenkins/secrets/initialAdminPassword

Copy the password and paste it into the setup screen.

5. Install Suggested Plugins

After unlocking Jenkins, you will be asked to install plugins. Choose the option to "Install suggested plugins" to automatically install a set of recommended plugins.

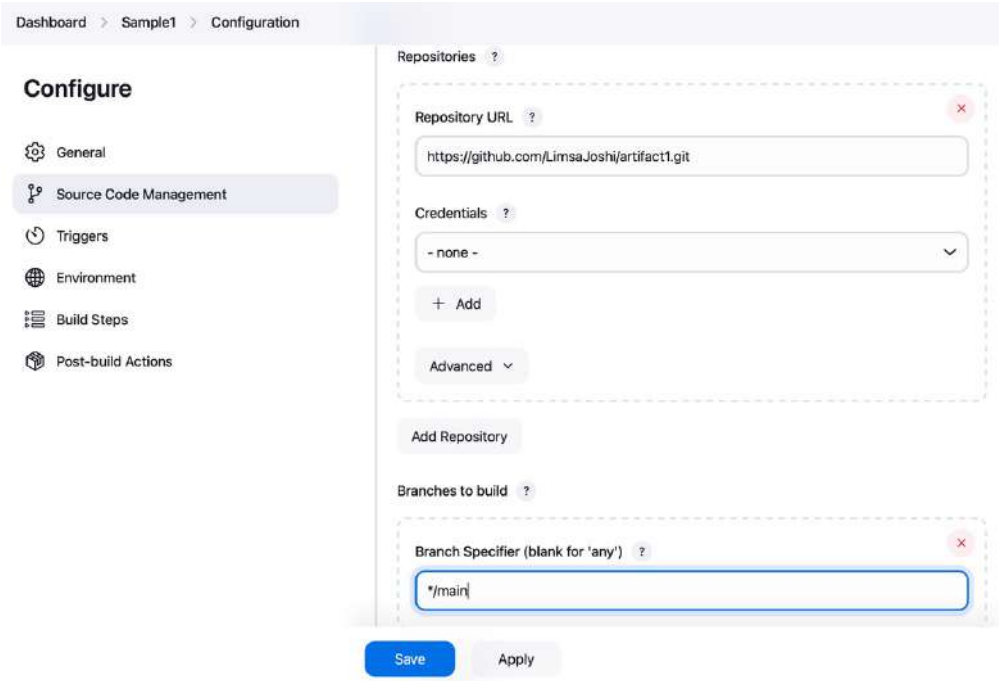
6. Install Additional Plugins

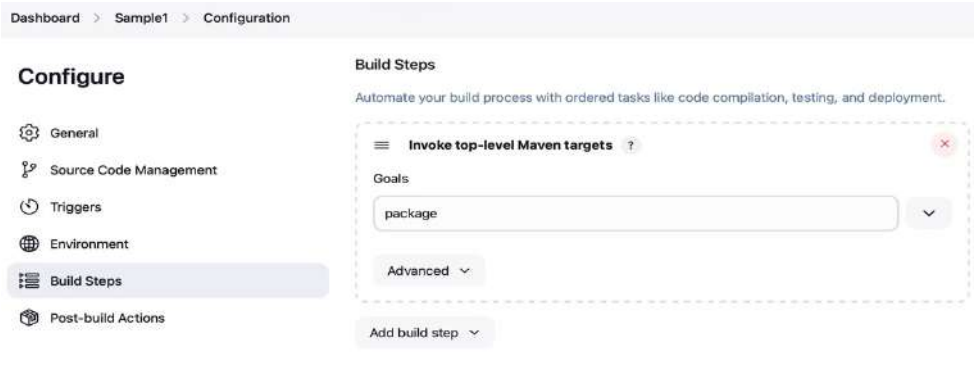
To install the additional plugins such as Copy Artifact, HTML Publisher, and Build Pipeline View:

- Go to “Manage Jenkins” □ “Manage Plugins”.
- Click on the “Available” tab.
- Search for the desired plugins: Copy Artifact, HTML Publisher, and Build Pipeline View.
- Check the box for each plugin and click “Install without restart”.

7. Create a New Jenkins Job

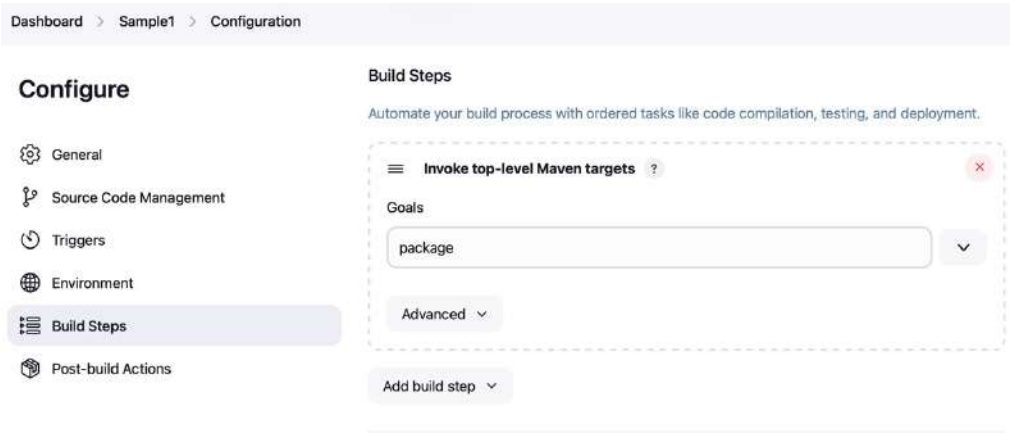
- From the Jenkins dashboard, click on “New Item”.
- Enter a name for your job and select “Freestyle project”, then click OK.
- Under “Source Code Management”, select “Git”.
- Enter the URL of your GitHub repository Check the branch
- In the “Build” section, click on “Add build step” and select “Invoke top-level Maven targets”.
- Enter your Maven goals and options (e.g., clean install).



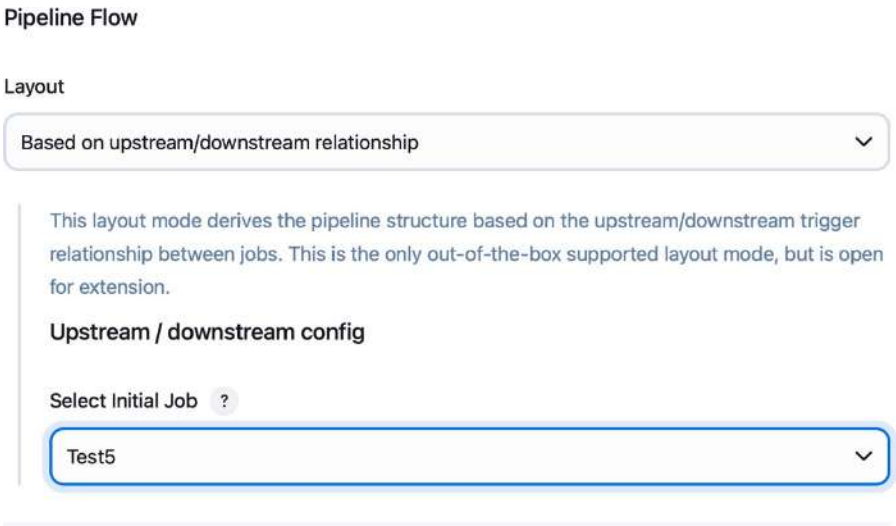


8. Create a Pipeline Using Multiple Jobs

- Install the “Pipeline” plugin if not already installed.
- Go to the Jenkins dashboard and click on “New Item”.
- Enter a name for your pipeline and select “Pipeline”, then click OK.

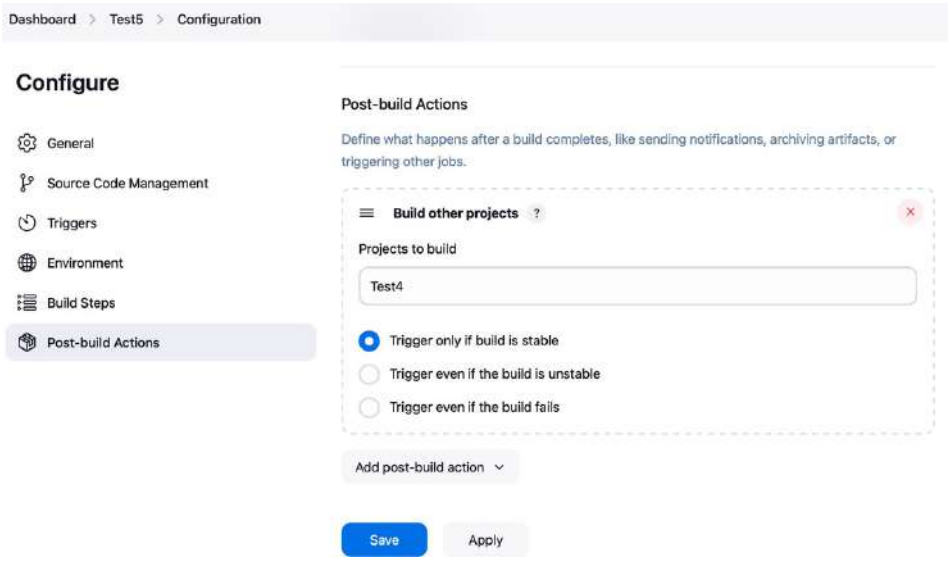


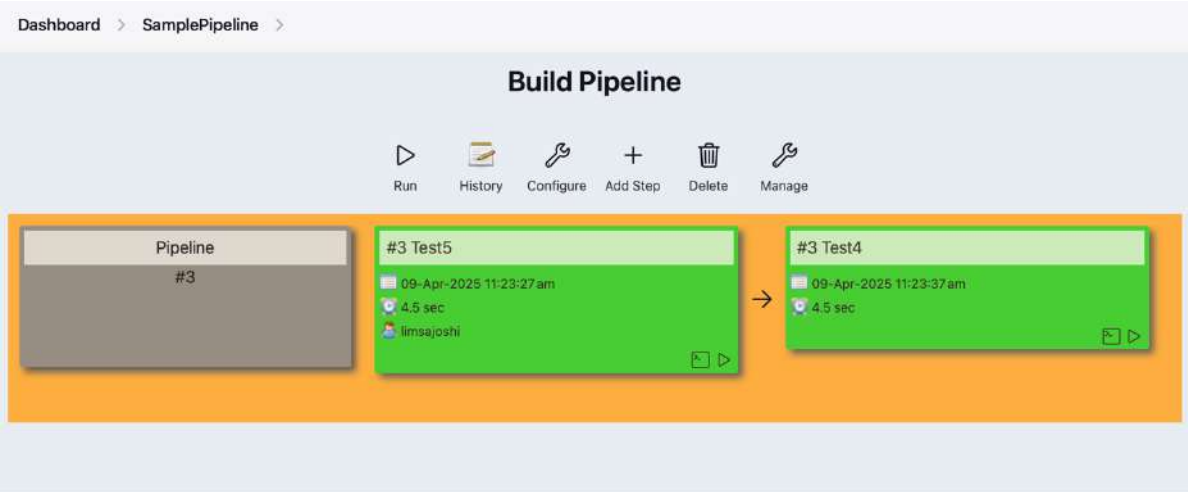
- In the configuration:
  1. Set "Select Initial Job" to the first job in your pipeline (e.g., Job1).
  2. Set the number of builds to display.
  3. Optionally configure filtering or display options.



- Click Apply → OK.
- Create the Initial Job (Job1 – Build)

  1. On the Jenkins dashboard, click "New Item".
  2. Enter a name (e.g., Job1).
  3. Select Freestyle project → Click OK.
  4. Under Build Steps, add your commands or build tool (Shell, Maven, Gradle, etc.).
  5. In the Post-build Actions, click "Add post-build action" → select "Build other projects".
  6. Enter the next job in sequence (e.g., Job2).
  7. Choose "Trigger only if build is stable".
  8. Click Save.

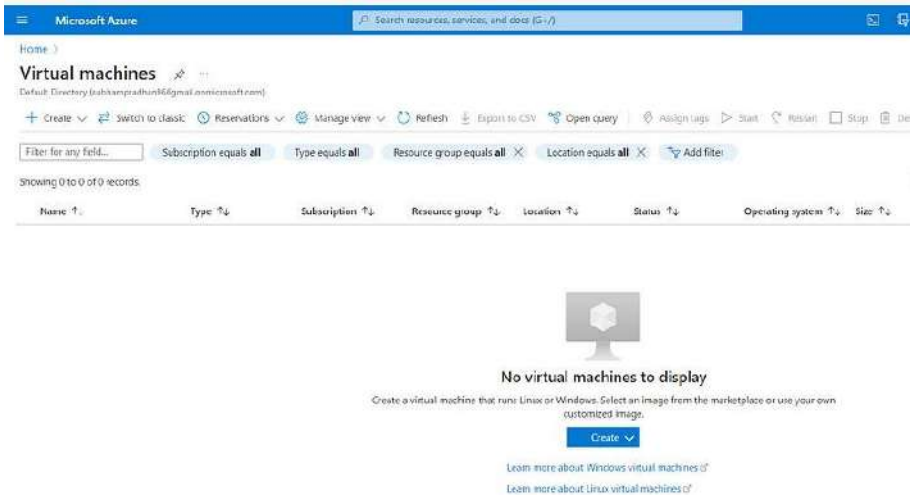

Dashboard > Test5 > Configuration
Configure
General
Source Code Management
Triggers
Environment
Build Steps
Post-build Actions
Post-build Actions
Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.
Build other projects
Projects to build
Test4
Trigger only if build is stable
Trigger even if the build is unstable
Trigger even if the build fails
Add post-build action
Save
Apply

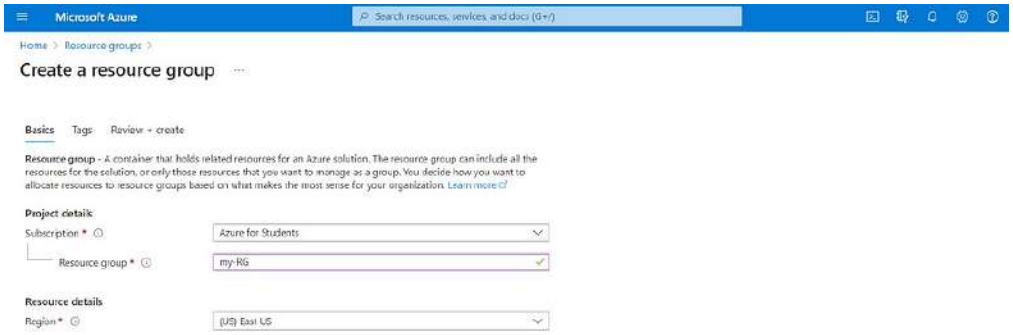

Dashboard > SamplePipeline >
Build Pipeline
Run
History
Configure
Add Step
Delete
Manage
Pipeline
#3
#3 Test5
09-Apr-2025 11:23:27 am
4.5 sec
limsajoshi
#3 Test4
09-Apr-2025 11:23:37 am
4.5 sec

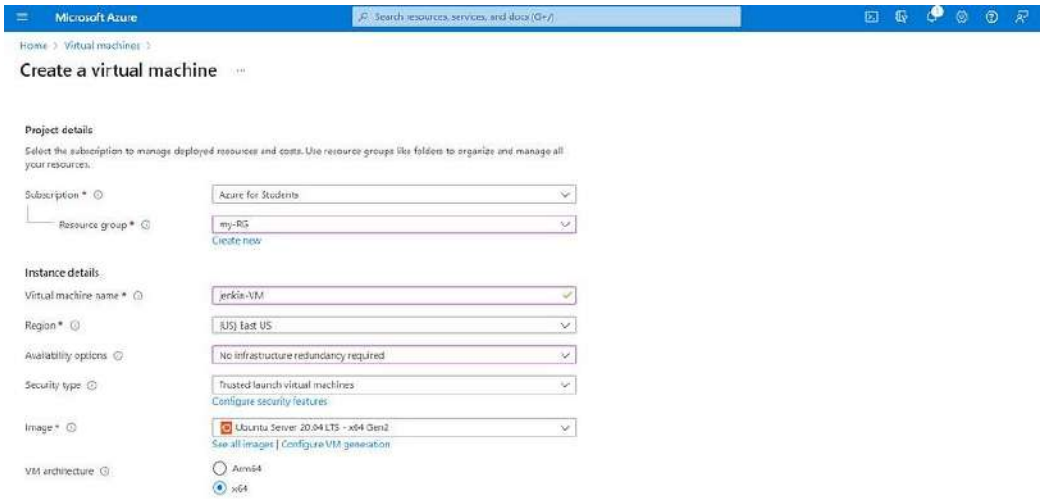
Ex No. 9 Install and configure Jenkins on Azure Linux VM

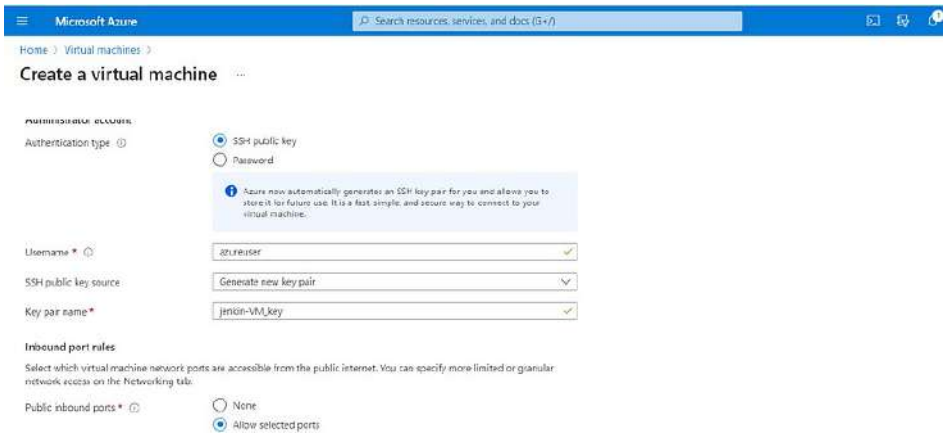
Step 1: To launch a virtual machine (VM) instance on Microsoft Azure, follow these steps:

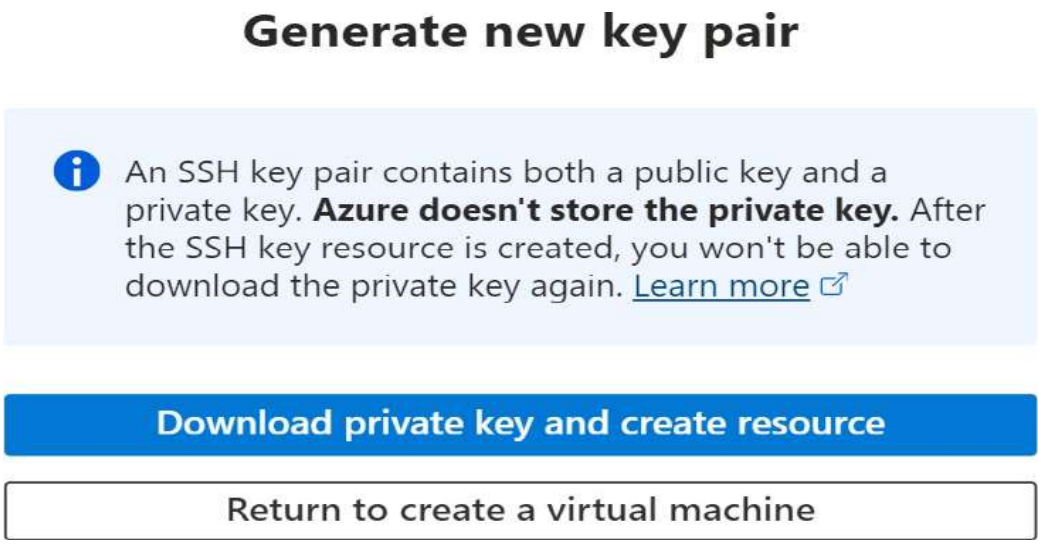
1. Sign in to the Azure portal: Go to https://portal.azure.com and sign in with your Azure account.
2. Create a new virtual machine:
3. In the Azure portal's left-hand menu, click on "Create a resource".
4. In the search box, type "Virtual Machine" and press Enter.
5. Select "Virtual machine" from the search results.






Microsoft Azure
Home > Virtual machines >
Create a virtual machine
Project details
Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.
Subscription *
Azure for Students
Resource group *
my-RG
Create new
Instance details
Virtual machine name *
jenkin-VM
Region *
(US) East US
Availability options
No infrastructure redundancy required
Security type
Trusted launch virtual machines
Configure security features
Image *
Ubuntu Server 20.04 LTS - x64 Gen2
See all images | Configure VM generation
VM architecture
Arm64
x64


Microsoft Azure
Home > Virtual machines >
Create a virtual machine
Authentication type
SSH public key
Password
Azure now automatically generates an SSH key pair for you and allows you to store it for future use. It is a fast, simple, and secure way to connect to your virtual machine.
Username *
azureuser
SSH public key source
Generate new key pair
Key pair name *
jenkin-VM_key
Inbound port rules
Select which virtual machine network ports are accessible from the public internet. You can specify more limited or granular network access on the Networking tab.
Public inbound ports *
None
Allow selected ports


Generate new key pair
An SSH key pair contains both a public key and a private key. Azure doesn't store the private key. After the SSH key resource is created, you won't be able to download the private key again. Learn more
Download private key and create resource
Return to create a virtual machine

Step 2: Connect to the VM using SSH using Azure CLI and Install Jenkins.

Pre-Requisites: Java (JDK)

Run the below commands to install Java and Jenkins

Install Java

sudo apt update

```
Microsoft Azure   Search resources, services, and docs (G+/)
Bash

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


Last login: Mon Apr 29 09:21:50 2024 from 20.235.209.48
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

limsa.25cs@licet.ac.in@JenkinsVM:~$ sudo apt update
Get:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease [270 kB]
Get:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
Get:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease [109 kB]
Get:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease [110 kB]
Get:5 http://azure.archive.ubuntu.com/ubuntu jammy/main amd64 Packages [1395 kB]
Get:6 http://azure.archive.ubuntu.com/ubuntu jammy/main Translation-en [510 kB]
Get:7 http://azure.archive.ubuntu.com/ubuntu jammy/main amd64 c-n-f Metadata [30.3 kB]
Get:8 http://azure.archive.ubuntu.com/ubuntu jammy/restricted amd64 Packages [129 kB]
Get:9 http://azure.archive.ubuntu.com/ubuntu jammy/restricted Translation-en [18.6 kB]
```

sudo apt install openjdk-17-jre

```
limsa.25cs@licet.ac.in@JenkinsVM:~$ sudo apt install openjdk-17-jre
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  adwaita-icon-theme alsa-topology-conf alsa-ucm-conf at-spi2-core ca-certificates-java dconf-gsetting
  fontconfig fontconfig-config fonts-dejavu-core fonts-dejavu-extra gsettings-desktop-schemas gtk-upda
  hicolor-icon-theme humanity-icon-theme java-common libasound2 libasound2-data libatk-bridge2.0-0 lib
  libatk-wrapper-java-jni libatk1.0-0 libatk1.0-data libatspi2.0-0 libavahi-client3 libavahi-common-da
  libcairo-gobject2 libcairo2 libcups2 libdatrie1 libdconf1 libdeflate0 libdrm-amdgpu1 libdrm-intel1 l
  libfontconfig1 libfontenc1 libgail-common libgail18 libgdk-pixbuf-2.0-0 libgdk-pixbuf2.0-bin libgdk-
  libgl1 libgl1-amber-dri libgl1-mesa-dri libglapi-mesa libglvnd0 libglx-mesa0 libglx0 libgraphite2-3
  libgtk2.0-common libharfbuzz0b libice6 libjbig0 libjpeg-turbo8 libjpeg8 liblcms2-2 libllvm15 libpang
  libpangoft2-1.0-0 libpciaccess0 libpcsclite1 libpixman-1-0 librsvg2-2 librsvg2-common libsensors-con
  libthai-data libthai0 libtiff5 libwebp7 libx11-xcb1 libxaw7 libxcb-dri2-0 libxcb-dri3-0 libxcb-glx0
  libxcb-render0 libxcb-shape0 libxcb-shm0 libxcb-sync1 libxcb-xfixes0 libxcomposite1 libxcursor1 libx
  libxi6 libxinerama1 libxkbfile1 libxmu6 libxpm4 libxrandr2 libxrender1 libxshmfence1 libxt6 libxtst6
  libxxf86vm1 openjdk-17-jre-headless session-migration ubuntu-mono x11-common x11-utils
```

```
limsa.25cs@licet.ac.in@JenkinsVM:~$ java -version
openjdk version "17.0.10" 2024-01-16
OpenJDK Runtime Environment (build 17.0.10+7-Ubuntu-122.04.1)
OpenJDK 64-Bit Server VM (build 17.0.10+7-Ubuntu-122.04.1, mixed mode, sharing)
```

Install Jenkins

curl -fsSL https://pkg.jenkins.io/debian/jenkins.io- 2023.key | sudo tee \
/usr/share/keyrings/jenkins-keyring.asc >
/dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins- keyring.asc] \
https://pkg.jenkins.io/debian binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list >
/dev/null

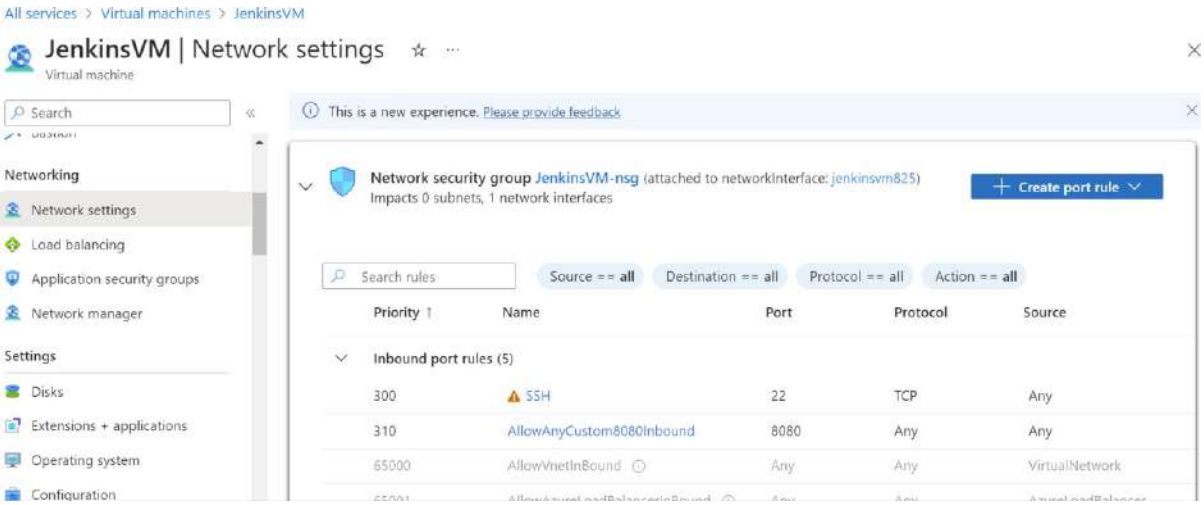
```
limsa.25cs@licet.ac.in@JenkinsVM:~$ curl -fsSL https://pkg.jenkins.io/debian/jenkins.io-2023.key | sudo tee   /usr/share/keyri
ins-keyring.asc > /dev/null
limsa.25cs@licet.ac.in@JenkinsVM:~$ echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]   https://pkg.jenkins.io/debi
y/ | sudo tee   /etc/apt/sources.list.d/jenkins.list > /dev/null
limsa.25cs@licet.ac.in@JenkinsVM:~$ sudo apt-get update
Hit:1 http://azure.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://azure.archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:3 http://azure.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:4 http://azure.archive.ubuntu.com/ubuntu jammy-security InRelease
Ign:5 https://pkg.jenkins.io/debian binary/ InRelease
Get:6 https://pkg.jenkins.io/debian binary/ Release [2044 B]
Get:7 https://pkg.jenkins.io/debian binary/ Release.gpg [833 B]
Get:8 https://pkg.jenkins.io/debian binary/ Packages [61.5 kB]
Get:9 https://packages.microsoft.com/repos/microsoft-ubuntu-jammy-prod jammy InRelease [3632 B]
Fetched 68.0 kB in 1s (71.4 kB/s)
Reading package lists... Done
```

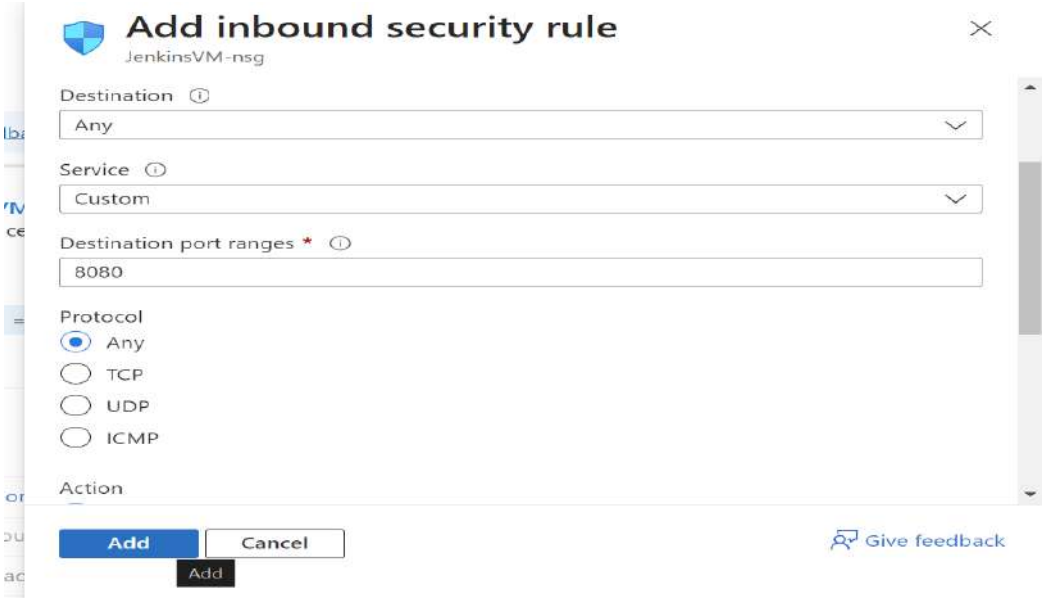
sudo apt-get update
sudo apt-get install Jenkins

```
limsa.25cs@licet.ac.in@JenkinsVM:~$ sudo apt-get install jenkins
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  net-tools
The following NEW packages will be installed:
  jenkins net-tools
0 upgraded, 2 newly installed, 0 to remove and 37 not upgraded.
Need to get 91.5 MB of archives.
After this operation, 94.2 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://azure.archive.ubuntu.com/ubuntu jammy/main amd64 net-tools amd64 1.60+git20181103.0eebece-1ubuntu5 [204 kB]
Get:2 https://pkg.jenkins.io/debian binary/ jenkins 2.455 [91.3 MB]
Fetched 91.5 MB in 8s (11.9 MB/s)
Selecting previously unselected package net-tools.
(Reading database ... 77655 files and directories currently installed.)
Preparing to unpack .../net-tools_1.60+git20181103.0eebece-1ubuntu5_amd64.deb ...
Unpacking net-tools (1.60+git20181103.0eebece-1ubuntu5) ...
Selecting previously unselected package jenkins.
Preparing to unpack .../archives/jenkins_2.455_all.deb ...
Unpacking jenkins (2.455) ...
```

By default, Jenkins will not be accessible to the external world due to the inbound traffic restriction by Azure. Open port 8080 in the inbound traffic rules as show below.

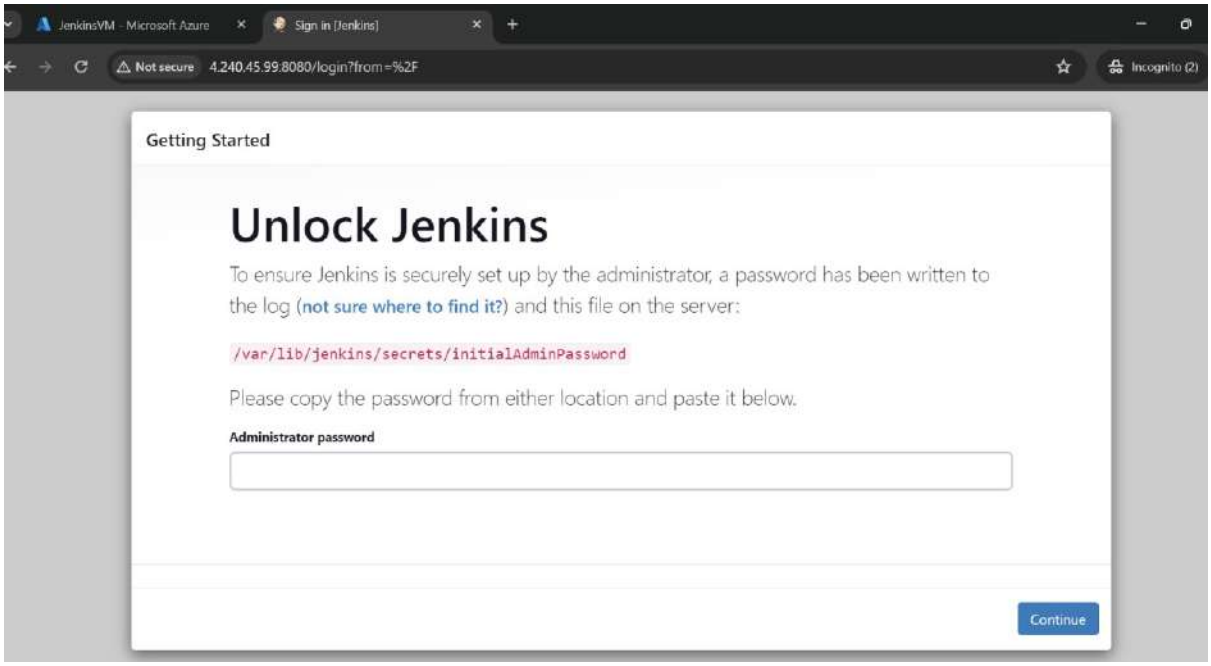
- Add inbound security rule
- Add inbound traffic rules as shown in the image (you can just allow 8080 as well, in my case, I allowed All traffic).

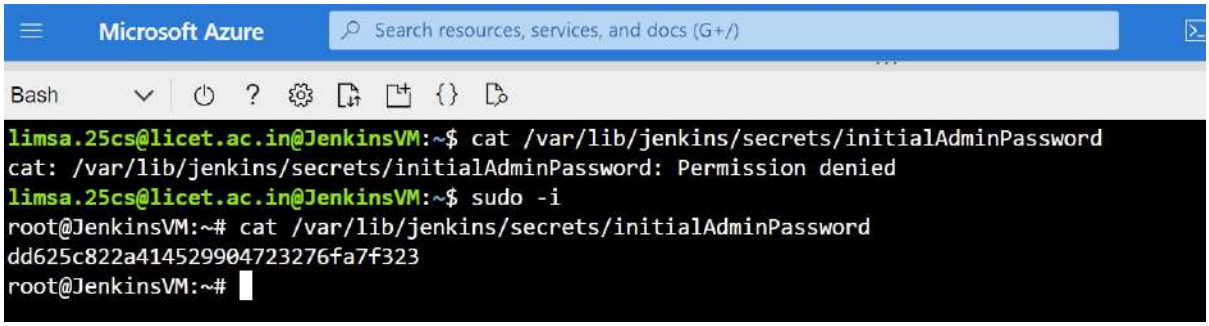




Step 3: Login to Jenkins using the below URL and configure Jenkins

http:/VM-public-ip/:8080 [You can get the VM-public-ip-address from your Azure console page]





Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log (not sure where to find it?) and this file on the server:

`/var/lib/jenkins/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

Continue

Getting Started

Customize Jenkins

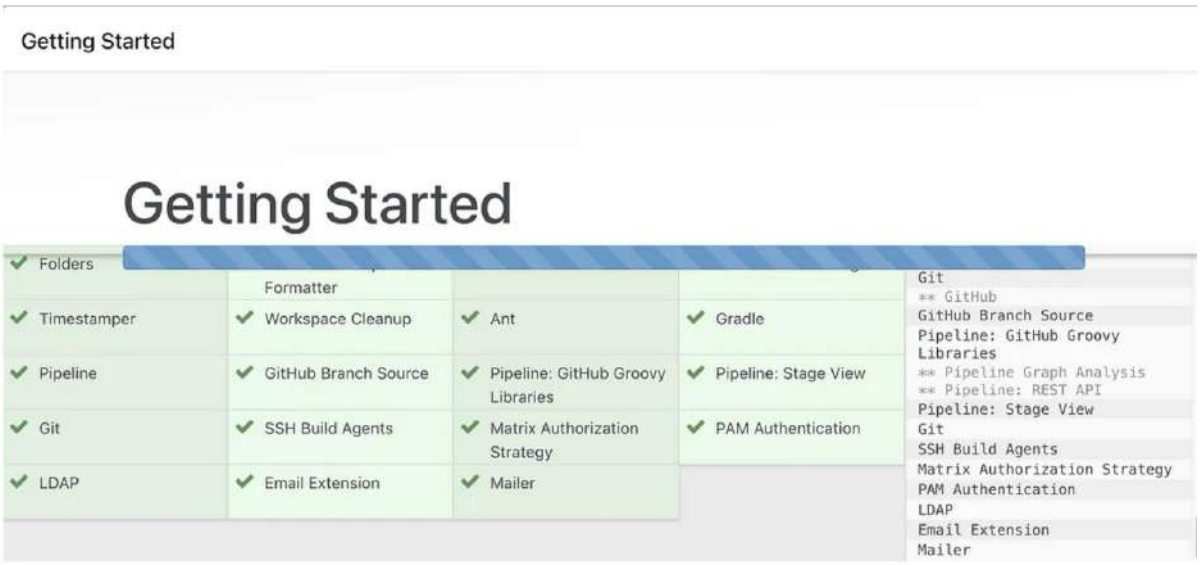
Plugins extend Jenkins with additional features to support many different needs.

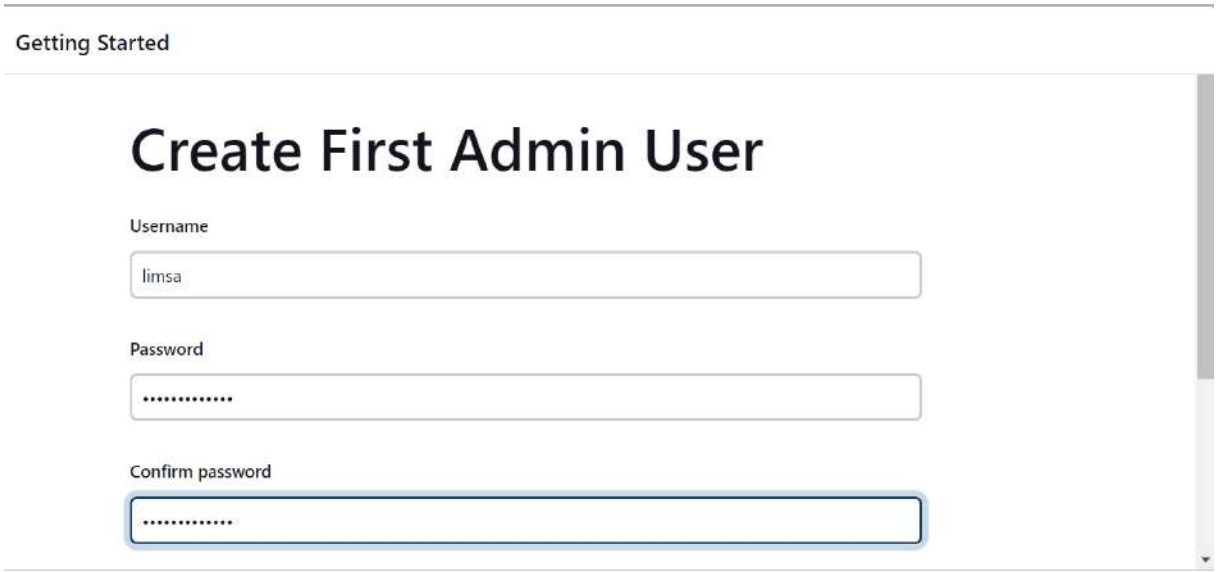
Install suggested plugins

Install plugins the Jenkins community finds most useful.

Select plugins to install

Select and install plugins most suitable for your needs.

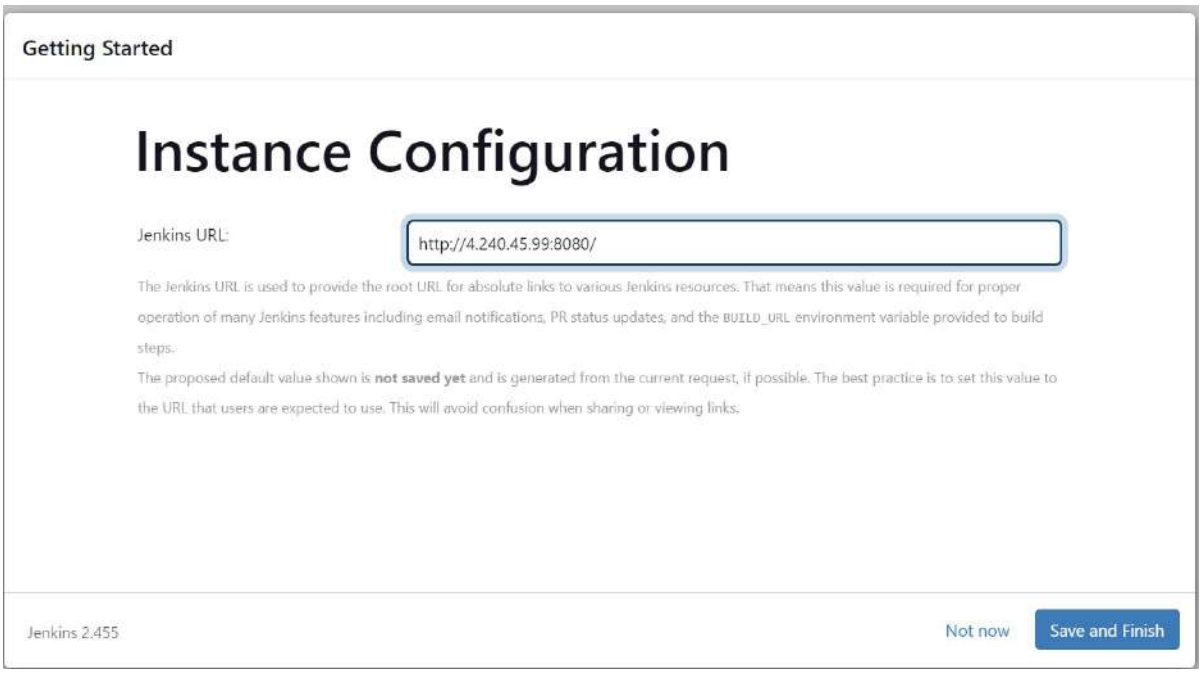

Getting Started
Getting Started
Folders
Formatter
Timestamper
Workspace Cleanup
Ant
Gradle
Pipeline
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
Pipeline: Stage View
Git
SSH Build Agents
Matrix Authorization Strategy
PAM Authentication
LDAP
Email Extension
Mailer
Git
** GitHub
GitHub Branch Source
Pipeline: GitHub Groovy Libraries
** Pipeline Graph Analysis
** Pipeline: REST API
Pipeline: Stage View
Git
SSH Build Agents
Matrix Authorization Strategy
PAM Authentication
LDAP
Email Extension
Mailer

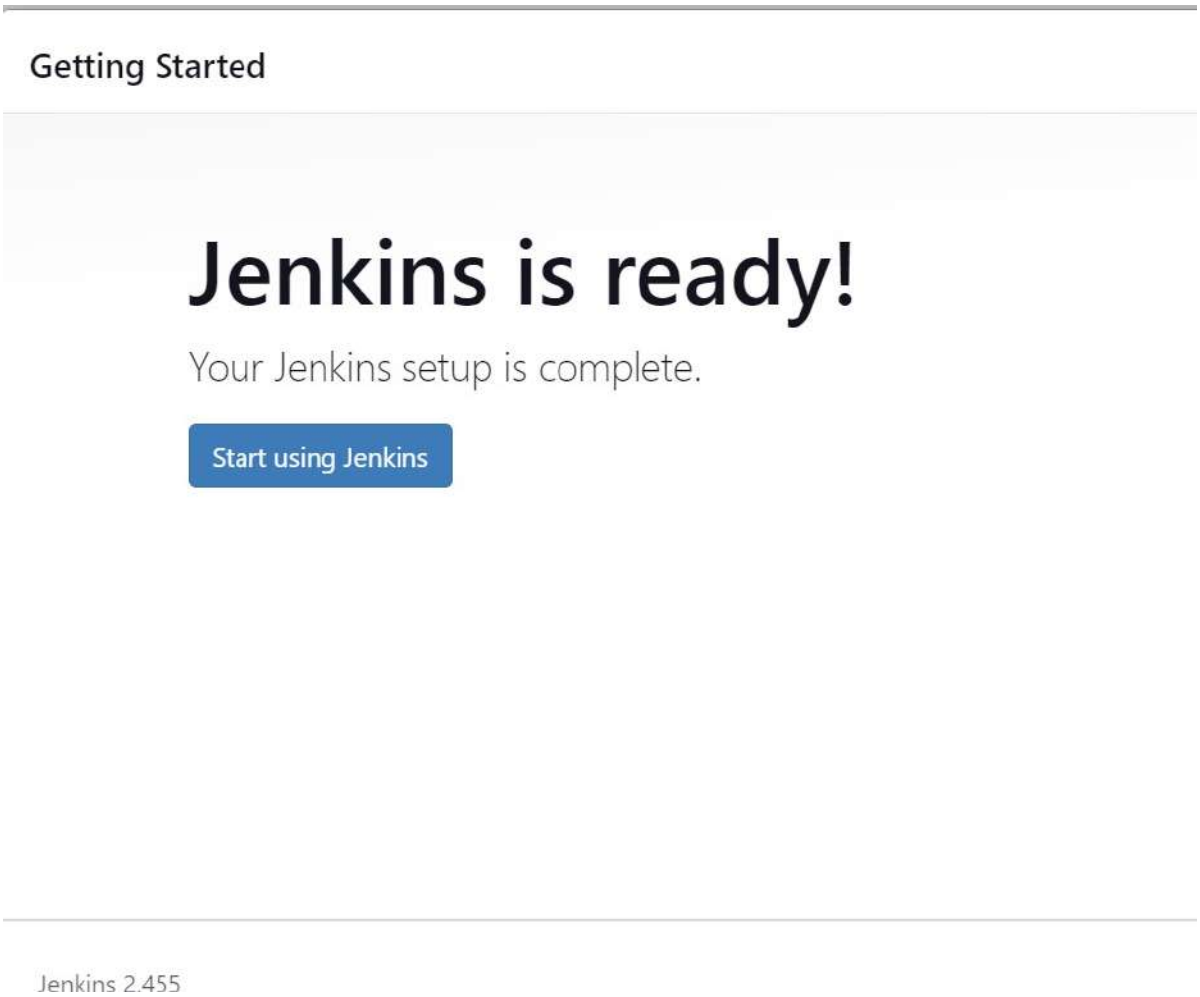

Getting Started
Create First Admin User
Username
limsa
Password
Confirm password

Jenkins 2.455
Skip and continue as admin
Save and Continue

Full name
Limsa Joshi J
E-mail address
limsa.25cs@licet.ac.in

Skip and continue as admin
Save and Continue


Getting Started
Instance Configuration
Jenkins URL:
http://4.240.45.99:8080/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.455
Not now
Save and Finish


Getting Started
Jenkins is ready!
Your Jenkins setup is complete.
Start using Jenkins
Jenkins 2.455

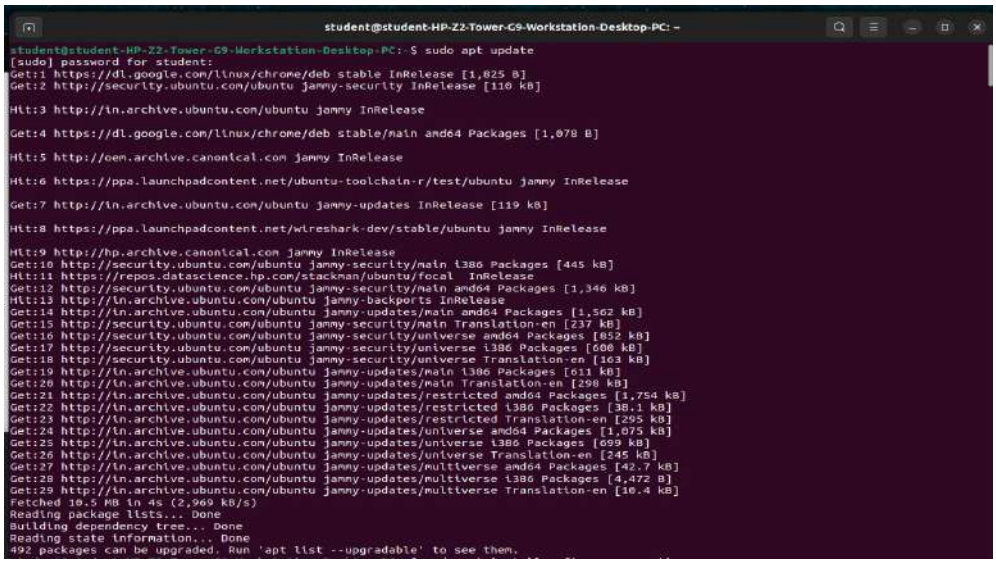
Ex No. 10 Ansible Installation and Create an Ansible playbook for a simple web application infrastructure

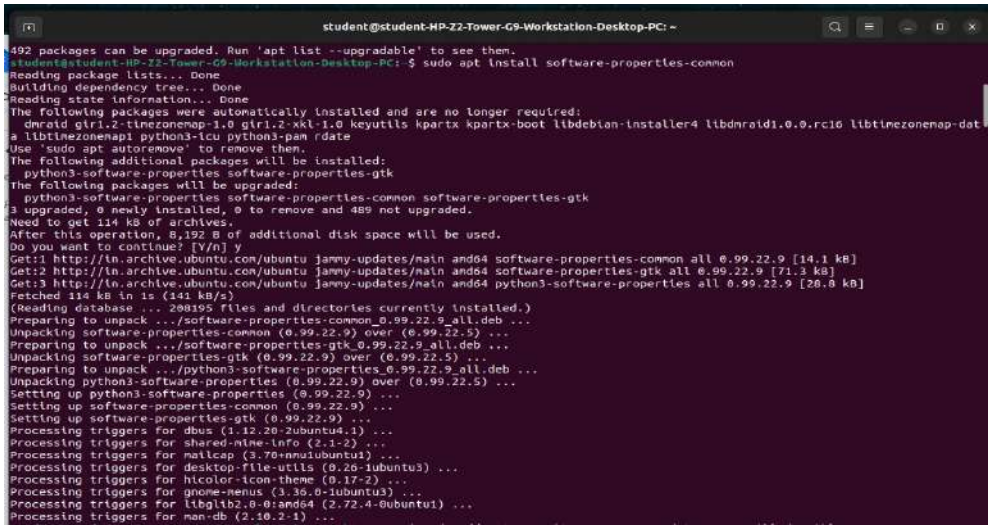
1. Install Ansible on your control node using the system package manager.

   $ sudo apt update
   $ sudo apt install software-properties-common
   $ sudo add-apt-repository --yes --update ppa:ansible/ansible
   $ sudo apt install ansible

   Error/Warning: If user not in the sudoers list, run the following commands

   - su
   - sudo usermod -aG sudo “username”
   - su “username”





```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ sudo add-apt-repository --yes --update ppa:ansible/ansible
Repository: 'deb https://ppa.launchpadcontent.net/ansible/ansible/ubuntu/ jammy main'
Description:
Ansible is a radically simple IT automation platform that makes your applications and systems easier to deploy. Avoid writing scripts
or custom code to deploy and update your applications— automate in a language that approaches plain English, using SSH, with no agents
 to install on remote systems.

http://ansible.com/

If you face any issues while installing Ansible PPA, file an issue here:
https://github.com/ansible-community/ppa/issues
More info: https://launchpad.net/~ansible/+archive/ubuntu/ansible
Adding repository.
Adding deb entry to /etc/apt/sources.list.d/ansible-ubuntu-ansible-jammy.list
Adding disabled deb-src entry to /etc/apt/sources.list.d/ansible-ubuntu-ansible-jammy.list
Adding key to /etc/apt/trusted.gpg.d/ansible-ubuntu-ansible.gpg with fingerprint 6125E2A8C77F2818FB7BD15B93C4A3FD7B9C367
Hit:1 https://dl.google.com/linux/chrome/deb stable InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease

Hit:3 http://oem.archive.canonical.com jammy InRelease

Hit:4 http://in.archive.ubuntu.com/ubuntu jammy InRelease

Hit:5 http://hp.archive.canonical.com jammy InRelease

Hit:6 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease
Get:7 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy InRelease [18.0 kB]
Hit:8 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease
Hit:9 https://ppa.launchpadcontent.net/ubuntu-toolchain-r/test/ubuntu jammy InRelease
Hit:10 https://repos.datascience.hp.com/stackman/ubuntu/focal  InRelease
Hit:11 https://ppa.launchpadcontent.net/wireshark-dev/stable/ubuntu jammy InRelease
Get:12 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy/main i386 Packages [1,128 B]
Get:13 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy/main amd64 Packages [1,128 B]
Get:14 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy/main Translation-en [752 B]
Fetched 21.1 kB in 1s (15.2 kB/s)
Reading package lists... Done
```

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~$ sudo apt install ansible
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  dmraid gir1.2-timezonemap-1.0 gir1.2-xkl-1.0 keyutils kpartx kpartx-boot libdeblan-installer4 libdmraid1.0.0.rc16 libtimezonemap-dat
a libtimezonemap1 python3-icu python3-pam rdate
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
  ansible-core python-babel-localedata python3-babel python3-jinja2 python3-jmespath python3-kerberos python3-ntlm-auth python3-reques
ts-kerberos python3-requests-ntlm python3-resolvelib python3-winrm
  python3-xmltodict sshpass
Suggested packages:
  python-jinja2-doc
The following NEW packages will be installed:
  ansible ansible-core python-babel-localedata python3-babel python3-jinja2 python3-jmespath python3-kerberos python3-ntlm-auth python
3-requests-kerberos python3-requests-ntlm python3-resolvelib
  python3-winrm python3-xmltodict sshpass
0 upgraded, 14 newly installed, 0 to remove and 489 not upgraded.
Need to get 22.9 MB of archives.
After this operation, 221 MB of additional disk space will be used.
Do you want to continue? [Y/n] y
Get:1 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 python-babel-localedata all 2.8.0+dfsg.1-7 [4,982 kB]
Get:2 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy/main amd64 ansible-core all 2.16.5-1ppa~jammy [1,032 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 python3-babel all 2.8.0+dfsg.1-7 [85.1 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu jammy-updates/main amd64 python3-jinja2 all 3.0.3-1ubuntu0.1 [108 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-resolvelib all 0.8.1-1 [23.6 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu jammy/main amd64 python3-jmespath all 0.10.0-1 [21.7 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-kerberos amd64 1.1.14-3.1build5 [23.0 kB]
Get:8 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-ntlm-auth all 1.4.0-1 [20.4 kB]
Get:9 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-requests-kerberos all 0.12.0-2 [11.9 kB]
Get:10 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-requests-ntlm all 1.1.0-1.1 [6,160 B]
Get:11 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-xmltodict all 0.12.0-2 [12.6 kB]
Get:12 https://ppa.launchpadcontent.net/ansible/ansible/ubuntu jammy/main amd64 ansible all 9.4.0-1ppa~jammy [16.5 MB]
Get:13 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 python3-winrm all 0.3.0-2 [21.7 kB]
Get:14 http://in.archive.ubuntu.com/ubuntu jammy/universe amd64 sshpass amd64 1.09-1 [11.7 kB]
Fetched 22.9 MB in 36s (644 kB/s)

Selecting previously unselected package python-babel-localedata.
(Reading database ... 208195 files and directories currently installed.)
Preparing to unpack .../00-python-babel-localedata_2.8.0+dfsg.1-7_all.deb ...
Unpacking python-babel-localedata (2.8.0+dfsg.1-7) ...
Selecting previously unselected package python3-babel.
Preparing to unpack .../01-python3-babel_2.8.0+dfsg.1-7_all.deb ...
Unpacking python3-babel (2.8.0+dfsg.1-7) ...
Selecting previously unselected package python3-jinja2.
Preparing to unpack .../02-python3-jinja2_3.0.3-1ubuntu0.1_all.deb ...
Unpacking python3-jinja2 (3.0.3-1ubuntu0.1) ...
```

2. Ensure Ansible is installed successfully

   ansible - - version

Error: UTF-8 encoding expected for ansible, run the following commands

- sudo nano /etc/default/locale
- Update the encoding to UTF-8
  LANG="en_US.UTF-8"
  LC_CTYPE="en.US.UTF-8"

(Or)

- Run the below command in terminal
  sudo update-locale LANG=en_US.UTF-8 LC_CTYPE=en_US.UTF-8

- Restart

3. Creating SSH Key - Creates a public/private SSH key pair for secure password less access.

```
ssh-keygen -t rsa -C "devops default"
ls -la .ssh
cat
.ssh/id_rsa.pub
cat .ssh/id_rsa
```

```
a12-60@a1260-ThinkStation-P330:~$ ssh-keygen -t rsa -C "devops default"
Generating public/private rsa key pair.
Enter file in which to save the key (/home/a12-60/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/a12-60/.ssh/id_rsa
Your public key has been saved in /home/a12-60/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:YAEh/GmvokUMQ5XM2q4eSbBgOLbfVRgurnY3nXQ+7T8 devops default
The key's randomart image is:
+---[RSA 3072]----+
| o=.+o..         |
|o .=  ..o        |
|Boo. ooo .       |
|=B..=....        |
|.o+. o .S. .     |
|..o.o o o + .    |
| oo+ + o o o .   |
| o+ o . .   o  E |
|oo .         ....|
+----[SHA256]-----+
```

```
a12-60@a1260-ThinkStation-P330:~$ ls -la .ssh
total 28
drwx------  2 a12-60 a12-60 4096 Apr 10 14:26 .
drwxr-x--- 22 a12-60 a12-60 4096 Apr 10 14:02 ..
-rw-------  1 a12-60 a12-60  568 Apr 10 14:26 authorized_keys
-rw-------  1 a12-60 a12-60 2655 Apr 10 14:14 id_rsa
-rw-r--r--  1 a12-60 a12-60  568 Apr 10 14:14 id_rsa.pub
-rw-------  1 a12-60 a12-60  978 Apr 10 14:26 known_hosts
-rw-r--r--  1 a12-60 a12-60  142 Apr 10 14:26 known_hosts.old
```

```
a12-60@a1260-ThinkStation-P330:~$ cat .ssh/id_rsa.pub
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDHOMOJ8AHYKgropa4JfR9D54ZVyeWdOFTYILtl1RZJBGtWCVIX7nQF7l7mDVa2lVmw67Tl3jUvEJdp9MbBs6ksM3LxlcP9v2zbEjolPUzGQjPRVQ
MNKwGZ9gqcPNJqEweUYStZgezwqAopcC4ZqzKEWYNyjj2RXJCDqbkthGOpYZRK0EV5BcUFZ0TXA+A1Jqs3dYCHIvdoOxR4PhyoouJJrS/y4FuabkzXrK50GGmebPhcgZazoYrZjqZ2TScWOsMJGQYI
7KVwPIIreU+uOhv69TqdRDcfn3jZZBtapK1ExpNZ2DG1TNHdwt87cX1lZ9tQAPEBEfNXzHE3TCeeUUoQSk7tT//d5tAgBaChBtG/s1dbZfFel9jV3aVcwE5k9UgkdB/Mx25hntwB434kmYLv+PE3ek
kobOxfkGN95wroGp7Nk20RXb/zpLWA6oIP+5RuvGPK4mLeunABodK4oSPRttfUTTEren/Mf/AAyfHFJ9vNqo9rdzB4KCzdhTxIw4E= devops default
```

```
a12-60@a1260-ThinkStation-P330:~$ cat .ssh/id_rsa
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAAACmFlczI1NiAAAAGYmNyeXB0AAAAGAAAABB/2Jq0gG
xILaK5GDjG/wsjAAAAEAAAAAEAAAGXAAAAB3NzaC1yc2EAAAADAQABAAABgQDHOMOJ8AHY
Kgropa4JfR9D54ZVyeWdOFTYILtl1RZJBGtWCVIX7nQF7l7mDVa2lVmw67Tl3jUvEJdp9M
bBs6ksM3LxlcP9v2zbEjolPUzGQjPRVQMNKwGZ9gqcPNJqEweUYStZgezwqAopcC4ZqzKE
WYNyjj2RXJCDqbkthGOpYZRK0EV5BcUFZ0TXA+A1Jqs3dYCHIvdoOxR4PhyoouJJrS/y4F
uabkzXrK50GGmebPhcgZazoYrZjqZ2TScWOsMJGQYI7KVwPIIreU+uOhv69TqdRDcfn3jZ
ZBtapK1ExpNZ2DG1TNHdwt87cX1lZ9tQAPEBEfNXzHE3TCeeUUoQSk7tT//d5tAgBaChBt
G/s1dbZfFel9jV3aVcwE5k9UgkdB/Mx25hntwB434kmYLv+PE3ekkobOxfkGN95wroGp7N
k20RXb/zpLWA6oIP+5RuvGPK4mLeunABodK4oSPRttfUTTEren/Mf/AAyfHFJ9vNqo9rdz
B4KCzdhTxIw4EAAAWQvdsSTDBW4Q5Hn36bG29S/ljmv8EW2XvZm86Lc17NsU6uPA2EXIHA
eulpFuZcEjHiGNAz553w0kQkjasB9/YH2TFv4BceDIw+dXg/TeJeTXKbslbLPj/SrD8oe4
AIjCLdRa4g4XLOk+y5OGNxlOwSrhlEvmUL5NlLU6KU+D9zXtCnYVTQe1cbF3o4Sm0i8ECl
BTESRUOc+YSSUQfgcbkLzIlTh90yw2VFMeKTzC2Z7olpCJDJNSlvC6spH9JllfGDvYt1GG
V+vTcj5Nk+qvp54vbG2BDhOgKae3zKuCY+SvMTCo/uBLtNFqO1qn8pRjCijj3XExJXX2Rd
POE7vRH1vOWRsCfxW9/yfNj1WBvZIcmAgm5b1QF21GhVPHuShLTpxeOYJfxp2MlRkxxSo7
/4ImW+B7omEuL81rCRHoswQsBlGOd6LGtNSO3ZwmuOTSu4TW55Zkq3nlZB3Jy/v6I39EC4
prAC10a01m6sA6UwcLH4Sn0Y3joLCUt3vKMneRYC8d4JQBl/LTp20cONz04q4KlueGGr3A
s9d6Baydqnklb17XG0q/aD/ryx2AthW4I/KHLZvlM4/NRXHJX3/7JE8QEchZQSqdZvO8sf
gJwgbfG4nv9H5qNDLYheQ8ux5BAB23V9xyNSpCkpto0rJ8aijzOz4jXdgneoUAU1jQ6HyJ
eklPpLiVi86QbvlwrNeT/t73C200GRHO6nMgouFaPX+Tuq7ZUA+FfkHnbXR8kyrwSvPrNO
BmPQ8FQ1ZHylzsHS0tzUYmQBWNoVR5NBxflQ8wsrdCSC2yYH7bruYM57StpcdK/GG8F3u
I9flKFCfdOC91tKZjWWJBrLyF3dbczqu5HTI9q+3gkEs/KoqUQBspgfITnY5CXqXzNpVCH5
s78jE0ILWe5jjtHEhIechSnvEkjH6EH25V2kZUmg3JVBDSDx9Cqgkmi6XP4s/Gpu7NAO0k
z1IhEFUD9sgQDQSNCom5baAVOpmS1pD+VKlpmFtlEaFcag7qQ1hgBUDDo8+uwHaN0kuzgk
hK7FWQ8guuqSk0ajncJt3LpjDCMVrekVdNDwnCtoGKqRecyBKD/l0JL4yNXjV9d4XWPrlu
vS3SUFzmC/PDHE7D93pJLRmxllEFXWKc+gSrkgQwAxDjVuN7lssC6ybdYulPXCyF++BU9X
Jw4rahFCOy/h0tV3uWLmI0kHUmbEuNZt0Jv+/QXHzt/XlwSX0+ltpypD5rOkrGEZt7Zvk2
C9/x/TJZyrX6PB0IqTPkh4gme1N2P9yK+nPTcVI98FPccgm5AqMmgfm1K0GlLQ5C7+gcH9
ZrosExfnvZobbSdel+UmDyToBXpKgc6kBjvPqeQQHKTZ1xunexWQVkUXwkvBNKN/U+27mU
lXaoTjm9N07Vg//10GfnQkkMZZy4vWIgNe9LR5BDDUH+gYbdDRj8G6Jv0ADZMsR2WDLflLF
rfmiroqHfdkeLoOc+pcQUQDr0b2BbTboTJMYOz9mWo5wf0qufeXyKDe/8D22R+jdaIdFXP
Ewo4R81cyur3SLb8bsCS2n+nrjH2hF5jOlJhUZsESu3laR2NOS01kpHZuTueuejBRc6e32
CemNk6yWLU+6EB3tOMEnbqcb6zBLXOBJVeeq1Qh2KdW0uSCH1XWqW32mEYRsU66Troy2Pc
lsBSfBgmoEcyyMqw96dufnQ5NdzPdAM5GbK4DKw2JdDMNsh6VYNKs0AudVGTz15etQbcm
0sMhBxeqjr1LICl0j+/MDYv0RuOpPyYsa3JH6lzcOKr9MWBKprgdMbzSBoLj2p7Suzne42
5VhEJtR7HX7Xef/PXOGm3+W7O9uolTJW3EV/Fkj8xTTqCqcWwRFYsFRevcKfjWgu+9n1Wz
DhM3LEk8lMpgEvr6uCEhhLIacJA=
-----END OPENSSH PRIVATE KEY-----
```

4. Copying SSH Key to server - Enables password less SSH login by copying the key to managed nodes.

ssh-copy-id -i ~/.ssh/id_rsa.pub "ipaddress"

```
a12-60@a1260-ThinkStation-P330: $ ssh-copy-id -i ~/.ssh/id_rsa.pub "192.168.1.160"
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/a12-60/.ssh/id_rsa.pub"
The authenticity of host '192.168.1.160 (192.168.1.160)' can't be established.
ED25519 key fingerprint is SHA256:NcgS7RFcaxEduvcaHevD6iAxihkc9YBtkV85dTaegCI.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? ^C/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
 out any that are already installed
/usr/bin/ssh-copy-id: 183: cannot create /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_tmp_id: Directory nonexistent
/usr/bin/ssh-copy-id: 189: cannot create /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: Directory nonexistent
grep: /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: No such file or directory
/usr/bin/ssh-copy-id: 202: cannot open /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: No such file
cat: /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_tmp_id: No such file or directory

/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system.
                (if you think this is a mistake, you may want to use -f option)
```

```
a12-60@a1260-ThinkStation-P330:~$ ls -la .ssh
total 28
drwx------  2 a12-60 a12-60 4096 Apr 10 14:26 .
drwxr-x--- 22 a12-60 a12-60 4096 Apr 10 14:02 ..
-rw-------  1 a12-60 a12-60  568 Apr 10 14:26 authorized_keys
-rw-------  1 a12-60 a12-60 2655 Apr 10 14:14 id_rsa
-rw-r--r--  1 a12-60 a12-60  568 Apr 10 14:14 id_rsa.pub
-rw-------  1 a12-60 a12-60  978 Apr 10 14:26 known_hosts
-rw-r--r--  1 a12-60 a12-60  142 Apr 10 14:26 known_hosts.old
```

```
a12-60@a1260-ThinkStation-P330:~$ cat .ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABgQDHOMOJ8AHYKgropa4JfR9D54ZVyeWdOFTYILti1RZJBGtWCVIX7nQF7l7mDVa2lVmw67Tl3jUvEJdp9MbBs6ksM3LxlcP9v2zbEjolPUzGQjPRVQ
MNKwGZ9gqcPNJqEweUYStZgezwqAopcC4ZqzKEWYNyjj2RXJCDqbkthGOpYZRK0EV5BcUFZ0TXA+A1Jqs3dYCHIvdoOxR4PhyoouJJrS/y4FuabkzXrK50GGmebPhcgZazoYrZjqZ2TScWOsMJGQYI
7KVwPIIreU+uOhv69TqdRDcfn3jZZBtapK1ExpNZ2DG1TNHdwt87cX1iZ9tQAPEBEfNXzHE3TCeeUUoQSk7tT//d5tAgBaChBtG/s1dbZfFel9jV3aVcwE5k9UgkdB/Mx25hntwB434kmYLv+PE3ek
kobOxfkGN95wroGp7Nk2ORXb/zpLWA6oIP+5RuvGPK4mLeunABodK4oSPRttfUTTEren/Mf/AAyfHFJ9vNqo9rdzB4KCzdhTxIw4E= devops default
```

5. Testing the new Key - Verifies successful login to remote servers without a password.

ssh -i ~/.ssh/"keyname" "ipaddress"

Error: port 22, Connection Refused, run the following commands

sudo apt install openssh-server
sudo systemctl restart sshd

```
a12-60@a1260-ThinkStation-P330: $ ssh-copy-id -i ~/.ssh/id_rsa.pub "192.168.1.160"
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/a12-60/.ssh/id_rsa.pub"
The authenticity of host '192.168.1.160 (192.168.1.160)' can't be established.
ED25519 key fingerprint is SHA256:NcgS7RFcaxEduvcaHevD6iAxihkc9YBtkV85dTaegCI.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? ^C/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter
 out any that are already installed
/usr/bin/ssh-copy-id: 183: cannot create /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_tmp_id: Directory nonexistent
/usr/bin/ssh-copy-id: 189: cannot create /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: Directory nonexistent
grep: /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: No such file or directory
/usr/bin/ssh-copy-id: 202: cannot open /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_output: No such file
cat: /home/a12-60/.ssh/ssh-copy-id.BXoz6WmA3p/popids_tmp_id: No such file or directory

/usr/bin/ssh-copy-id: WARNING: All keys were skipped because they already exist on the remote system.
                (if you think this is a mistake, you may want to use -f option)
```

6. Connecting to Server - ssh "Ip address"

```
a12-60@a1260-ThinkStation-P330:~$ ssh 192.168.1.160
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 6.5.0-26-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

 * Introducing Expanded Security Maintenance for Applications.
   Receive updates to over 25,000 software packages with your
   Ubuntu Pro subscription. Free for personal use.

     https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

111 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update
```

7. Create Inventory File - Lists all managed servers in a structured format for Ansible to use.

```
sudo nano inventory

[servers]
192.168.1.219
192.168.1.218
```

8. Ansible Command to connect with repository provided in the inventory ansible all --key-file ~/.ssh/id_rsa -i inventory -m ping

Error: If permission Denied, run – "chmod 600 ~/.ssh

Inventory Testing - ansible all -i inventory -m ping

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/Ansible$ ansible all --key-file ~/.ssh/ansible -i inventory -m ping
192.168.1.219 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
192.168.1.218 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

9. Creating playbook named playbook.yml

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/Ansible$ sudo nano playbook.yaml
```

playbook.yml

```
---
# This playbook prints a simple debug message
- name: Echo
  hosts: servers
  connection: local
  tasks:
  - name: Print debug message
    debug:
      msg: Hello, world!
```

Executing playbook - ansible-playbook -i inventory playbook.yml

```
student@student-HP-Z2-Tower-G9-Workstation-Desktop-PC:~/Ansible$ ansible-playbook -i inventory playbook.yaml

PLAY [Echo] ***********************************************************************************************************
*************************************************************************************************************************

TASK [Gathering Facts] ************************************************************************************************
*************************************************************************************************************************
ok: [192.168.1.218]
ok: [192.168.1.219]

TASK [Print debug message] ********************************************************************************************
*************************************************************************************************************************
ok: [192.168.1.219] => {
    "msg": "Hello, world!"
}
ok: [192.168.1.218] => {
    "msg": "Hello, world!"
}

PLAY RECAP ************************************************************************************************************
*************************************************************************************************************************
192.168.1.218              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
192.168.1.219              : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```